

ENERGY DASHBOARD · SMART ENERGY CLOUD

Backend Design

Models, services, APIs and realtime data flow

Turning the browser-only demo into a live, multi-tenant energy-monitoring platform

Document	Technical design · v1.1
Date	9 October 2026
Prepared for	Febino Solutions · Energy Dashboard team
Applies to	Front end v2.4.0 (branch features/feature1)
Status	Draft for review

Contents

1. Overview	4
1.1 Goals	4
1.2 Scope	4
1.3 How the pieces fit	4
2. Technology stack	5
3. Data models	6
3.1 Entity overview	6
3.2 Platform and tenants	6
3.3 Identity and access	8
3.4 Sites, devices and meters	10
3.5 Telemetry (time-series)	14
3.6 Tariffs, shifts and schedules	15
3.7 Alerts and notifications	17
3.8 Audit, exports and device events	19
4. Page-by-page field reference	21
4.1 Corrections summary	21
4.2 Every app page — top bar	22
4.3 Sign-in and registration pages	23
4.4 Dashboard	24
4.5 Energy Meters (meters.html)	25
4.6 Meter page (meter.html)	26
4.7 Device Management (devices.html, device.html)	27
4.8 Connections (connections.html)	30
4.9 Analytics (analytics.html)	30
4.10 Alerts (alerts.html)	30
4.11 Shift management (shifts, schedule, instances)	31
4.12 Users (users.html)	32
4.13 Roles & Permissions (roles.html)	33
4.14 Settings (settings.html)	33
4.15 Control Center (tenants, approvals, tenant page)	35
5. Services (backend modules)	35
6. REST API	39
6.1 Conventions	39
6.2 Example: /auth/me	42
7. Realtime data flow	42

7.1 Device ingestion over MQTT	42
7.2 Processing steps	43
7.3 Stream consumers	43
7.4 WebSocket protocol (browser)	43
7.5 Online, idle and offline	44
8. Alert engine	44
8.1 Rule JSON (as saved by the UI)	44
8.2 Parameters and operators	44
8.3 State machine	45
8.4 Evaluation details	45
9. Calculations	45
10. Security	46
11. Connecting the existing front end	46
12. Project structure	47
12.1 Environment variables	47
12.2 Local environment (docker-compose)	47
13. Build plan	48
13.1 Definition of done for 'realtime'	48
13.2 Testing	48

1. Overview

Today the Energy Dashboard runs entirely in the browser. Tenants, users, sessions and one-time codes live in `localStorage` (`assets/js/store.js`), the meter and device registries are static files (`meter-data.js`, `device-data.js`) and live values are simulated with a random generator. This document specifies the backend that replaces all of that, so the same pages show **real readings from real meters, in real time**, shared by every user of a tenant.

1.1 Goals

- **Realtime:** a reading taken by a gateway appears on the Dashboard and meter pages within about 1–2 seconds (15 s polling stays the device's choice).
- **Multi-tenant:** every organisation (Org ID such as FEBINO) is isolated; the platform admin (Control Center) approves, suspends and limits tenants.
- **Same front end:** the existing pages keep their layout; only the data layer changes (an API client replaces EDStore internals).
- **Trustworthy numbers:** energy from meter registers, ToD tariff cost, 15-minute maximum demand, single-phase and three-phase handled correctly.
- **Alerts that act:** the alert rules already designed in the UI (guided or expression, hold time, shifts/ToD windows, escalation) run on the server 24×7.
- **Secure by default:** server-side passwords, sessions, permissions, field-level visibility and tenant limits — nothing trusted from the browser.

1.2 Scope

In scope	Out of scope (later)
Auth, tenants, users, invites, roles, field permissions, limits	Billing / payment gateway for plans
Sites, devices, meters & sub-meters, connection flow, virtual meters	Mobile apps (the API supports them later)
Device ingestion (MQTT, HTTPS, Modbus TCP polling), time-series storage	Machine-learning forecasting (solar forecast uses a simple baseline first)
Live WebSocket feed, analytics, tariffs & cost, shifts	Remote meter configuration writes (read-only first)
Alert engine, notifications (in-app, email, SMS/WhatsApp, webhook), exports, audit	

1.3 How the pieces fit

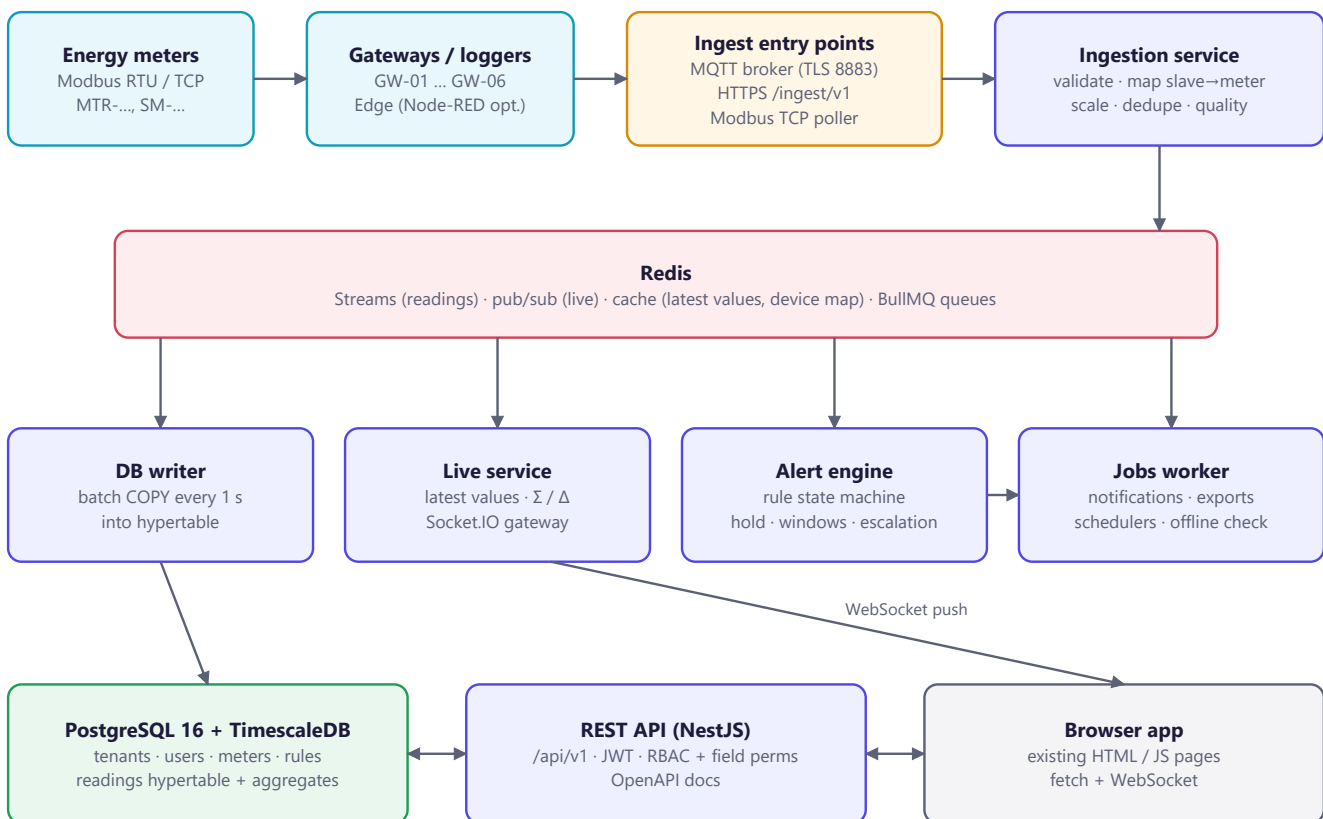


Figure 1 — System architecture. Arrows show the direction data moves. Everything except the browser runs on the server side.

The design separates the **hot path** (readings arriving every few seconds) from the **request path** (people using the app). Readings go device → ingest → Redis Stream, and three independent consumers write them to the database, push them to browsers and evaluate alerts. If one consumer slows down, the others are unaffected and nothing is lost, because each consumer group keeps its own position in the stream.

2. Technology stack

Recommended choices are listed first. The design does not depend on a specific vendor; the alternatives work with the same models and APIs.

Layer	Recommended	Why	Alternative
Runtime / language	Node.js 22 LTS + TypeScript	Same language as the front end; one team can work on both	Python (FastAPI), Go
API framework	NestJS	Modules map 1:1 to the services below; guards, DI, WebSocket gateway, OpenAPI built in	Express / Fastify
Relational + time-series DB	PostgreSQL 16 + TimescaleDB	One database for business data and readings; hypertables, compression, continuous aggregates, retention	PostgreSQL + InfluxDB
ORM / migrations	Prisma (tables) + SQL migrations (Timescale)	Typed queries; Timescale features need raw SQL	TypeORM, Drizzle
Cache, streams, queues	Redis 7 + BullMQ	Streams for readings, pub/sub for live fan-out, queues for jobs, rate limits	RabbitMQ / Kafka at large scale
Device messaging	EMQX (MQTT 5, TLS)	Per-device auth/ACL via HTTP hook to the API; handles thousands of gateways	Mosquitto (small sites)
Browser realtime	Socket.IO + Redis adapter	Rooms per tenant/meter, auto-reconnect, works behind proxies	Native WebSocket / SSE
Email / SMS / WhatsApp	SMTP or AWS SES; MSG91 / Twilio	Alert and OTP delivery	Any provider behind the Notifications service

Layer	Recommended	Why	Alternative
File storage	S3-compatible (MinIO in dev)	Generated exports and reports	Local disk (single server)
Observability	pino logs, Prometheus, Grafana, OpenTelemetry	Ingest lag, alert latency, API errors	ELK, Datadog
Deployment	Docker Compose → Kubernetes	Same images from laptop to production; Nginx with TLS in front	Single VM with PM2

3. Data models

All business tables carry `tenant_id` and are protected by PostgreSQL row-level security (section 9). Primary keys are UUIDs; the human codes the UI already shows (MTR-1001, SM-1001-01, GW-01, AR-7F2, EMP-014) are kept as separate code columns, unique per tenant. Every table also has `created_at` and `updated_at` (timestampz), omitted below for brevity. Times are stored in UTC and shown in the tenant's time zone (default Asia/Kolkata).

3.1 Entity overview

Area	Tables
Platform	platform_admins, tenants, tenant_status_history, tenant_limits, audit_log
Identity & access	users, user_sites, invites, auth_sessions, otp_codes, roles, permissions, role_permissions, field_permissions
Assets	sites, devices, device_credentials, register_maps, cost_centres, meters, energy_targets, virtual_meter_members, connection_flows
Telemetry	readings (hypertable), readings_1m / readings_15m / readings_1h / readings_1d (continuous aggregates), meter_live (Redis)
Operations	tariffs, tariff_instances, shifts, shift_schedules, shift_assignments, shift_instances
Alerts	alert_rules, alert_rule_state, alerts, alert_events, notifications, notification_preferences
Other	export_jobs, device_events, device_messages, device_commands, publish_destinations, publish_attempts

Key relationships: a **tenant** has many sites, users, devices and meters. A **meter** may have a parent meter (that makes it a sub-meter) and is read by exactly one **device** at one Modbus slave address. A sub-meter's device can differ from its parent's (the dashed "Sub-meter → device" wire on the Connections page). **Alert rules** belong to a meter and may also cover its sub-meters. **Readings** reference a meter and are partitioned by time.

3.2 Platform and tenants

platform_admins Control Center accounts — a separate realm from tenant users

Column	Type	Notes
id	uuid PK	
username	citext unique	e.g. superadmin
name, email	text	
password_hash	text	argon2id
mfa_secret	text null	TOTP; required in production
last_login_at	timestampz null	

tenants One organisation (the Org ID users type at sign-in)

Column	Type	Notes
id	varchar(12) PK	Org ID: chosen slug (FEBINO) or generated 8-char code (HX4R7TQM)
name	text	legal name, Febino Solutions Pvt Ltd
slug	citext unique	Workspace URL, febino; <code>^[a-z0-9][a-z0-9-]{1,29}\$</code>
industry	text	Manufacturing plant, Data centre, Solar / renewable plant ...

Column	Type	Notes
gstin	varchar(15) null	hidden from some roles by field permissions
country, timezone, address, city	text	timezone default Asia/Kolkata
discom	text	utility, e.g. BESCOM, TANGEDCO
contract_demand_kva	integer	for maximum-demand checks
meter_range	text	1-10 11-50 51-200 200+ (sets default meter limit)
plan	enum	starter growth enterprise
billing_cycle	enum	monthly yearly
contact_email, phone	text	
status	enum	pending active suspended
status_reason	text null	e.g. Subscription invoice overdue
suspended_until	timestamptz null	suspension lifts itself at this time (scheduler)
approved_at, approved_by	timestamptz, uuid null	platform admin who approved
currency	char(3)	INR USD AED
energy_unit	enum	kWh MWh (display only; storage is always kWh)
co2_factor	numeric(5,3)	kg CO2 per kWh, default 0.820
default_tariff	numeric(8,2)	₹/kWh used when no ToD slot matches (UI demo uses 8)
deleted_at	timestamptz null	soft delete; data purged after 30 days

tenant_status_history Every approve / reject / suspend / reactivate

Column	Type	Notes
id	uuid PK	
tenant_id	FK tenants	
from_status, to_status	enum	
reason	text null	
until	timestamptz null	for temporary suspension
by_admin_id	FK platform_admins null	null = system (auto-lift)
at	timestamptz	

tenant_limits Overrides of the plan's default limits (Control Center → Limits)

Column	Type	Notes
tenant_id	FK, PK part	
key	enum, PK part	users meters subMeters devices sites tariffs shifts schedules alertRules roles
value	integer	0 = unlimited
reason	text null	why the admin changed it
set_by, set_at	uuid, timestamptz	

Default limits come from the plan (starter: 25 sub-meters, 2 devices, 1 site, 5 tariffs, 3 shifts, 3 schedules, 10 alert rules, 3 roles; growth: 250 / 20 / 5 / 20 / 10 / 20 / 50 / 10; enterprise: unlimited), users from the plan's seats (5 / 50 / 1000) and meters from meter_range (10 / 50 / 200 / 500). A row in tenant_limits overrides the default.

3.3 Identity and access

users A person's account inside one tenant (the same email may exist in several tenants)

Column	Type	Notes
id	uuid PK	
tenant_id	FK tenants	
emp_id	varchar(16)	EMP-014; unique per tenant, auto-next
first_name, last_name	text	
email	citext	unique per tenant
username	citext unique	unique across the platform (sign-in by username works without guessing tenant)
mobile	varchar(20) null	for OTP sign-in and SMS alerts
role_id	FK roles	
shift_id	FK shifts null	Shift A / B / C / General
department, designation	text null	
status	enum	invited active inactive suspended
two_factor	boolean	require OTP after password
language	varchar(8)	en-IN hi ta (My profile)
timezone	text null	IANA name; null = tenant's
password_hash	text null	null until the invite is accepted
last_active_at	timestampz null	shown as "12 min ago"
invited_by	FK users null	

user_sites Optional: restrict a user to some sites

Column	Type	Notes
user_id	FK, PK part	
site_id	FK, PK part	no rows = all sites

invites Invitation links (accept-invite.html?token=...)

Column	Type	Notes
id	uuid PK	
user_id	FK users	
token_hash	char(64) unique	SHA-256 of the random token; the token itself is only in the email
invited_by	FK users	
expires_at	timestampz	7 days
accepted_at	timestampz null	

auth_sessions Signed-in devices (Settings → Your active sessions)

Column	Type	Notes
id	uuid PK	
realm	enum	tenant platform
user_id / admin_id	uuid	one of them
tenant_id	FK null	current tenant (changes on switch-tenant)
refresh_token_hash	char(64)	rotated on every refresh; reuse of an old one revokes the session
remember	boolean	true: 30 days, false: 12 hours (platform: 8 hours)

Column	Type	Notes
ip, user_agent, device_label	text	"Chrome on Windows · Bengaluru"
last_used_at, expires_at, revoked_at	timestampz	

otp_codes One-time codes (verify email, OTP sign-in, password reset, login approval)

Column	Type	Notes
id	uuid PK	
purpose	enum	verify_email login_otp reset_password login_approval
target	text	email or mobile
user_id / tenant_id	uuid null	
code_hash	text	6 digits, hashed
expires_at	timestampz	10 minutes
attempts	smallint	locked after 5 wrong tries
consumed_at	timestampz null	

roles Built-in and custom roles

Column	Type	Notes
id	uuid PK	
tenant_id	FK tenants	
name	text	Tenant Admin, Energy Manager, Supervisor, Operator, Technician, Viewer, Auditor, or custom
description	text	
is_system	boolean	Type: Built-in Custom; built-in roles cannot be deleted; Tenant Admin cannot be restricted
landing_page	text	Default landing page: dashboard meters analytics alerts

permissions Catalogue of permission keys (Roles → Permissions tab)

Column	Type	Notes
key	varchar(48) PK	module.action, e.g. meters.export, alerts.acknowledge
module	enum	dashboard, meters, devices, connections, analytics, alerts, shifts, users, roles, settings
action	text	view, create, edit, delete, export, approve, acknowledge ...
description	text	What does this allow?
tenant_id	FK null	null = platform-defined; tenants may add custom keys

role_permissions Which keys a role has (the Permission matrix is a view over this)

Column	Type	Notes
role_id	FK, PK part	
permission_key	FK permissions, PK part	

field_permissions Field-level visibility per role (field-perms.js)

Column	Type	Notes
tenant_id, role_id	FK, PK part	
module, field	text, PK part	e.g. meters.cost, devices.mac, users.mobile, settings.gstin
can_view, can_edit	boolean	edit implies view; no row = full access

3.4 Sites, devices and meters

sites A location with its own utility connection

Column	Type	Notes
id	uuid PK	
tenant_id	FK	
name	text	Plant 1 · Bengaluru
address	text	
utility, consumer_no	text	DISCOM and service number
contract_demand_kva	integer	
default_tariff	enum	tod flat
flat_rate	numeric(8,2) null	₹/kWh when default_tariff = flat
timezone	text null	falls back to the tenant's
status	enum	active inactive

device_models Hardware catalogue — real gateways / loggers (hw-catalog.js)

Column	Type	Notes
id	uuid PK	
name	text unique	Moxa MGate MB3170, Schneider Link150, Schneider Com'X 510, Siemens SIMATIC IOT2050, Teltonika TRB245, Milesight UG65
maker, kind	text	Moxa · Modbus TCP ↔ RTU gateway
ethernet, serial_ports, cellular, wireless, io	text null	as published by the manufacturer
protocols	text[]	Modbus TCP, Modbus RTU, Modbus ASCII, LoRaWAN
uplinks	text[]	modbus_tcp_poll mqtt_push https_push
default_port	integer null	502, 443, 8883
power	text	12–48 VDC
notes, field_node	text null	e.g. Milesight UC501 node per meter
is_global	boolean	shared library; tenants may add their own

meter_models Hardware catalogue — real energy meters (hw-catalog.js)

Column	Type	Notes
id	uuid PK	
name	text unique	Schneider PM5560, PM2220, EM6400NG, iEM3255, Secure Elite 440, Fronius Smart Meter TS 65A-3, Deep Sea DSE7320 MKII
maker, kind	text	Schneider Electric · Power quality meter (PowerLogic PM5000)
meter_class	enum	basic multi pq dc dg — default class for meters of this model
accuracy	text	Class 0.2S (IEC 62053-22)
comms	text	2 × Ethernet (Modbus TCP) · 1 × RS-485 (Modbus RTU)

Column	Type	Notes
wiring	text[]	3P4W, 3P3W, 1P2W — the form only offers these
inputs, extras	text null	CT 1 A / 5 A secondary · harmonics to the 63rd
register_map_id	FK register_maps null	null = load the manufacturer's register list first
is_global	boolean	

Meter classes decide which readings a meter sends and the UI shows (telemetry tabs, log columns, payload keys):

basic kW, kWh · **multi** kW, V, I, PF, Hz, kWh · **pq** multi + THD V and THD I · **dc** DC V, I, kW, kWh (no PF / Hz) · **dg** kW, V, I, PF, Hz, fuel level, run hours, kWh.

devices Gateways and data loggers (Device Management)

Column	Type	Notes
id	uuid PK	
tenant_id, site_id	FK	
code	varchar(16)	GW-01; unique per tenant
name	text	Main LT Panel Gateway
type	enum	modbus_tcp_gateway energy_server edge_nodered edge_4g lorawan_gateway
device_model_id	FK device_models	the real product (interfaces, protocols, default port)
serial	text	hidden from Operator/Viewer by default
location	text	
ip, port, mac	inet, int, macaddr	
link	enum	ethernet wifi lte
field_protocol	text	Modbus RTU (RS-485), Modbus TCP, LoRaWAN
uplink	enum	mqtt_push https_push modbus_tcp_poll
polling_s	integer	15, 30, 60
serial_std	text null	RS-485 2-wire RS-485 4-wire RS-232; null when no serial line
baud	integer null	4800–115200; only for a serial field bus
parity	enum null	none even odd
data_bits, stop_bits	smallint null	8/7, 1/2 — must match every meter on the line
timeout_ms, retries	integer	Modbus reply timeout (default 1000) and retries (default 2)
time_sync	enum	ntp server
firmware, firmware_available	text	an update is offered when they differ
installed_on	date	
status	enum	online weak offline maintenance (derived, see 6.6)
signal_pct	smallint null	
last_seen_at	timestampz null	"4 s ago"

device_credentials The device token — how a device proves who it is

Column	Type	Notes
device_id	FK PK	
client_id	text unique	{orgId},{deviceCode}, e.g. FEBINO.GW-07 (MQTT username)
token_hash	char(64) unique	SHA-256 of the token; the token itself is never stored
token_prefix	char(8)	"edt_" + first 4 chars — to recognise a token in logs
token_last4	char(4)	shown masked: edt_••••a91F

Column	Type	Notes
created_at, created_by	timestampz, FK users	generated together with the device
rotated_at, rotated_by	timestampz null	Edit device → Regenerate token
last_used_at	timestampz null	
cert_fingerprint	text null	optional mutual TLS instead of a token

Device token: edt_ + 40 random base-62 characters (about 238 bits), generated by the server with a cryptographic random generator when the device is created. It is returned **once** in the response of POST /devices and shown in the Add device dialog; the user must confirm it was copied before the device is created. Only the SHA-256 hash is stored (a high-entropy random token needs no slow hash), so it can never be shown again — only replaced. Regenerating invalidates the old token at once and disconnects the device (the broker is told to kick the client). The device sends it as the **MQTT password** (username = client ID) or as Authorization: Bearer edt_... for HTTPS push. Polled devices (Modbus TCP) do not need one, but get one for heartbeats.

register_maps Modbus register layout per meter model (for polled devices)

Column	Type	Notes
id	uuid PK	
name	text unique	Schneider PM5xxx, Schneider iEM3xxx, Secure Elite 4xx, DSE GenComm ...
registers	jsonb	[[{param:'kw', address:3060, fc:3, type:'float32', order:'ABCD', scale:1, unit:'kW'}, ...] — 1-based as in the maker's list
source, verified_at	text, timestampz null	document the addresses came from; set when checked on a real meter
is_global	boolean	shared library; tenants may add their own

cost_centres For allocating energy cost (Analytics, sub-meters)

Column	Type	Notes
id	uuid PK	
tenant_id	FK	
code	varchar(16)	CC-PROD, CC-UTIL, CC-ADMIN, CC-IT, CC-FAC; unique per tenant
name	text	Production, Utilities ...

meters Energy meters and sub-meters (one table, parent_id makes a sub-meter)

Column	Type	Notes
id	uuid PK	
tenant_id, site_id	FK	
code	varchar(20)	MTR-1001 for meters, SM-1001-01 for sub-meters; unique per tenant
parent_id	FK meters null	null = main meter (counts toward 'meters' limit); set = sub-meter ('subMeters' limit)
name	text	Main Incomer, Feeder A · Production Line 1
kind	enum	consumer generator (solar, DG) — generators show 'Generating'
category	enum	incomer hvac lighting process utility other
cost_centre_id	FK cost_centres null	mainly for sub-meters
meter_model_id	FK meter_models null	the real product (accuracy class, comms, supported wiring)
location	text	
meter_class	enum	basic multi pq dc dg — required; defaults from the model
energy_direction	enum	import (incoming feeder) export (outgoing feeder) — required when the meter is created
serial_no	text null	from the meter label

Column	Type	Notes
wiring	enum	3P4W 3P3W 1P2W — limited to the model's list
pt_ratio	text null	e.g. 11000/110; null for LT (415 V direct)
commissioned_on, calibration_due	date null	commissioning record
phase	smallint	1 = single-phase 2-wire 230 V L-N; 3 = three-phase 4-wire 415 V L-L
nominal_v	numeric	230 or 415 (for formulas and limits)
ct_primary, ct_secondary	integer null	CT 400/5 → 400, 5; null = direct (e.g. Direct 63 A)
direct_a	integer null	rating for direct-connected sub-meters
rated_kw	numeric null	for Load % = kW / rated
device_id	FK devices null	the device that reads it (may differ from the parent's device)
modbus_slave_id	smallint null	1–247; unique together with device_id
register_map_id	FK register_maps null	defaults from model
register_offset	integer null	multi-channel meters: channel's base register (sub-meter form)
mode	enum	active maintenance disabled — the only status a user sets
icon, color	text	UI only (i-cpu, --cyan)
status	enum	online idle offline maintenance generating standby — derived from readings + mode, never edited
archived_at	timestamptz null	keeps history when a meter is removed

energy_targets Monthly target & allocation (Analytics → Edit target & allocation)

Column	Type	Notes
id	uuid PK	
tenant_id, meter_id	FK	
month	date	first day of month; unique (meter_id, month)
target_kwh	numeric(12,1)	> 0
cost_centre_id	FK null	
allocation_pct	numeric(5,2)	0–100; share charged to the cost centre
tariff_mode	enum	tod flat
notes	text null	reason for target change
set_by	FK users	

virtual_meter_members Virtual meters (Σ nodes on the Connections canvas)

Column	Type	Notes
virtual_id	FK meters (kind=virtual)	a virtual meter is a meters row without device
member_id	FK meters	a meter, sub-meter or another virtual meter
sign	smallint	+1 add, -1 subtract (e.g. Incomer – Solar)

connection_flows Saved and deployed versions of the Connections canvas

Column	Type	Notes
id	uuid PK	
tenant_id	FK	
version	integer	increments on each deploy
nodes	jsonb	[[{id, type: submeter meter virtual device, name, ref, x, y, props}]]

Column	Type	Notes
wires	jsonb	[{from, to, kind: direct sub link}]
is_draft	boolean	one draft per tenant; deploy creates a new version
deployed_by, deployed_at	uuid, timestampz	

Deploying a flow validates the same rules the canvas enforces (a sub-meter feeds at most one meter and one device; a meter reports through one device; devices have no outputs; no cycles among virtual meters) and then, in one transaction, updates `meters.device_id`, `meters.parent_id` and `virtual_meter_members`, and refreshes the device→meter map in Redis so ingestion routes new readings correctly within seconds.

3.5 Telemetry (time-series)

readings TimescaleDB hypertable — one row per meter per reading

Column	Type	Notes
time	timestampz	meter/gateway timestamp (UTC); chunk interval 1 day
tenant_id	varchar(12)	for RLS and per-tenant queries
meter_id	uuid	PK = (meter_id, time) — duplicates are ignored
kw, kvar, kva	real	total active / reactive / apparent power
pf	real	total power factor (−1...1)
v1, v2, v3	real	phase-to-neutral volts; 1φ meters fill only v1
v12, v23, v31	real null	line-to-line volts (3φ only)
i1, i2, i3, i_n	real	phase and neutral amps; 1φ meters fill only i1
hz	real	frequency
thd_v, thd_i	real	% total harmonic distortion (pq meters; per phase in thd_v1–3, thd_i1–3)
thd_v1, thd_v2, thd_v3, thd_i1, thd_i2, thd_i3	real null	per-phase THD for the Trends harmonics charts
fuel_pct, run_hours	real null	DG set controllers only
kwh_imp, kwh_exp	double precision	cumulative energy registers (import / export)
kvarh	double precision null	
quality	smallint	bit flags: 1 estimated, 2 out-of-range, 4 register reset, 8 late
received_at	timestampz	server time (for ingest-lag monitoring)

```

-- Readings hypertable (SQL migration)
CREATE TABLE readings (
  time timestampz NOT NULL, tenant_id varchar(12) NOT NULL, meter_id uuid NOT NULL,
  kw real, kvar real, kva real, pf real,
  v1 real, v2 real, v3 real, v12 real, v23 real, v31 real,
  i1 real, i2 real, i3 real, i_n real, hz real, thd_v real, thd_i real,
  thd_v1 real, thd_v2 real, thd_v3 real, thd_i1 real, thd_i2 real, thd_i3 real, fuel_pct real, run_hours real,
  kwh_imp double precision, kwh_exp double precision, kvarh double precision,
  quality smallint NOT NULL DEFAULT 0, received_at timestampz NOT NULL DEFAULT now(),
  PRIMARY KEY (meter_id, time)
);
SELECT create_hypertable('readings', 'time', chunk_time_interval => interval '1 day');
ALTER TABLE readings SET (timescaledb.compress, timescaledb.compress_segmentby = 'meter_id');
SELECT add_compression_policy('readings', interval '3 days');
SELECT add_retention_policy('readings', interval '90 days');      -- raw data kept 90 days

-- 15-minute aggregate: demand intervals, energy, power quality
CREATE MATERIALIZED VIEW readings_15m WITH (timescaledb.continuous) AS
SELECT time_bucket('15 minutes', time) AS bucket, tenant_id, meter_id,
  avg(kw) AS kw_avg, max(kw) AS kw_max, avg(kva) AS kva_avg, avg(pf) AS pf_avg,
  avg(thd_v1) AS thd_v1, avg(thd_v2) AS thd_v2, avg(thd_v3) AS thd_v3, avg(thd_i1) AS thd_i1, avg(thd_i2) AS thd_i2, avg(thd_i3) AS thd_i3,
  min(least(v1, coalesce(v2, v1), coalesce(v3, v1))) AS v_min,
  max(greatest(v1, coalesce(v2, v1), coalesce(v3, v1))) AS v_max,
  max(thd_i) AS thd_i_max,
  last(kwh_imp, time) - first(kwh_imp, time) AS kwh,
  last(kwh_exp, time) - first(kwh_exp, time) AS kwh_exp,
  count(*) AS samples
FROM readings GROUP BY bucket, tenant_id, meter_id;
SELECT add_continuous_aggregate_policy('readings_15m',
  start_offset => interval '2 hours', end_offset => interval '1 minute', schedule_interval => interval '1 minute');

```

The same pattern creates **readings_1m** (charts for today, kept 1 year), **readings_1h** and **readings_1d** (kept for ever; daily energy, peak kW, minimum/maximum voltage, average PF). Charts pick the aggregate by range: up to 6 hours → raw/1m, up to 7 days → 15m, up to 3 months → 1h, longer → 1d.

Sizing: 50 meters × one reading every 15 s = 288,000 rows/day (about 43 MB raw, roughly 4–5 MB after compression). A 500-meter tenant is about 2.9 M rows/day — well within a single TimescaleDB node.

Live cache (Redis, not a table): `live:{tenant}:{meterId}` holds the latest reading plus derived values (load %, imbalance, today's kWh and cost, Σ of sub-meters and Δ unmetered). The Dashboard and meter pages read it for their first paint; after that they receive updates over the WebSocket.

3.6 Tariffs, shifts and schedules

tariffs Time-of-Day tariff slots T1–T5 (Settings → Tariff slabs)

Column	Type	Notes
id	uuid PK	
tenant_id, site_id	FK (site null = all sites)	
slot	varchar(4)	T1 ... T5
name	text	Off-peak (night), Morning peak, Evening peak ...
start_time, end_time	time	22:00 → 06:00 crosses midnight
days	smallint	bit mask Mon=1 ... Sun=64 (Mon – Sat = 63)
energy_rate	numeric(8,2)	₹ per kWh, e.g. 6.80, 9.20, 9.60
demand_charge	numeric(8,2)	₹ per kVA of maximum demand per month
valid_from, valid_to	date	keeps expired slabs (FY 2025-26) for history
status	derived	current if today is within valid_from...valid_to, otherwise expired (not stored)

tariff_instances Each run of a ToD slot (Energy cost → Instances)

Column	Type	Notes
id	uuid PK	
tenant_id, site_id	FK	
code	varchar(20)	TI-1008-T2; TI-1005-T1E = Monday's early T1 after Sunday flat
tariff_id	FK tariffs	the slab in force when it ran (rate is copied below)
starts_at, ends_at	timestamp tz	T1 22:00 → 06:00 is one run, counted on the day it starts
status	enum	upcoming running closed
rate	numeric(8,2)	₹/kWh snapshot
energy_kwh, cost, peak_kw	numeric null	updated live while running, final when closed

Generating instances: the scheduler creates the next 2 days of tariff instances from the current slabs at midnight (Mon–Sat T1–T5, Sunday one Flat run; Saturday's T1 stops at 24:00 and Monday gets an early T1 00:00–06:00). The live service adds each reading's energy to the running instance; when it ends, the totals are recomputed from readings_15m and the status becomes closed. Instances, not raw readings, feed the cost tiles and the By-slot report.

shifts Shift definitions (Shift page)

Column	Type	Notes
id	uuid PK	
tenant_id	FK	
code, name	text	A / Shift A
start_time, end_time	time	end ≤ start means it ends the next day
break_min	smallint	0 30 45 60
count_on	enum	start_day end_day — which day an overnight shift belongs to
supervisor_id	FK users null	
grace_min	smallint	0–60 late-arrival grace
energy_target_kwh	numeric null	per shift run
status	enum	active inactive
color	text	amber orange violet cyan green

shift_schedules Repeating plans (Schedule page)

Column	Type	Notes
id	uuid PK	
tenant_id	FK	
name	text	
pattern	jsonb	weekly or rotation: [{day/offset, shift_id}]
shift_id	FK	the form's Shift
days	smallint	Repeats on (bit mask) — the days a shift runs come from its schedules
pattern	jsonb null	rotations later: [{offset, shift_id}]
starts_on, ends_on	date	Effective from / Until
status	enum	active paused

shift_assignments Who works which shift (Schedule → Assign staff to shift)

Column	Type	Notes
id	uuid PK	
tenant_id, user_id, shift_id	FK	
schedule_id	FK null	
from_date, to_date	date	to ≥ from; one user cannot have overlapping assignments

shift_instances Each actual run of a shift (Instance page)

Column	Type	Notes
id	uuid PK	
tenant_id, shift_id, schedule_id	FK	
code	varchar(20)	SI-1008-A
starts_at, ends_at	timestampz	planned times, overnight resolved
actual_start, actual_end	timestampz null	Planned / actual column
staff_count	smallint	from shift_assignments
production_units	numeric null	entered by supervisor; kWh / unit = energy_kwh / production_units
status	enum	planned running closed
energy_kwh, cost, peak_kw, avg_pf	numeric null	snapshot written when the instance closes

3.7 Alerts and notifications

alert_rules Per-meter rules (meter page → Alert rules; same shape the UI already saves)

Column	Type	Notes
id	uuid PK	
tenant_id	FK	
meter_id	FK null	null for tenant-wide rules (Alerts page)
code	varchar(10)	AR-7F2
name	text	1–80 chars
severity	enum	critical warning info
enabled	boolean	
scope	enum	meter subs meter_and_subs all all_meters all_subs
sub_ids	uuid[]	selected sub-meters ([] = all)
evaluate	enum	live 15m 1h shift day — live checks every reading; others check the window when it closes
cond	jsonb	{mode: guided expression, match: all any, rows:[{param, op, v1, v2}], expr, hold}
active	jsonb	{mode: always shifts slots custom, shifts, slots, from, to, days}
notify	jsonb	{channels: app email sms whatsapp webhook, roles, users, emails, repeat, resolve, escalate}
compiled	jsonb	server-validated form of cond (parsed expression, variable list)
created_by	FK users	

alert_rule_state Where each rule stands for each meter it watches (also cached in Redis)

Column	Type	Notes
rule_id, meter_id	PK	
state	enum	ok pending firing
since	timestampz	when the current state began
last_value	jsonb	values that made it true
alert_id	FK alerts null	the open alert while firing
last_notified_at, escalated_at	timestampz null	

alerts The Alert log

Column	Type	Notes
id	uuid PK	
tenant_id	FK	
rule_id	FK null	null for system alerts (device offline, firmware, budget)
meter_id, device_id	FK null	
title	text	THD above 8%
severity	enum	
category	enum	threshold power_quality energy device communication info
value_text, limit_text	text	9.1% / 8%
status	enum	open acknowledged resolved
assigned_to	FK users null	Update alert → Assign to
raised_at, acknowledged_at, resolved_at	timestampz	
acknowledged_by, resolved_by	FK users null	null = auto-resolved
note	text null	Resolution note

alert_events Timeline of an alert (raised, notified, escalated, acknowledged, resolved)

Column	Type	Notes
id	bigserial	
alert_id	FK	
type	text	
detail	jsonb	
at	timestampz	

notifications Every message sent, and the bell-icon inbox

Column	Type	Notes
id	uuid PK	
tenant_id	FK	
user_id	FK null	null for plain email addresses / webhooks
alert_id	FK null	
channel	enum	app email sms whatsapp webhook
to	text	
title, body	text	

Column	Type	Notes
status	enum	queued sent failed
error	text null	
sent_at, read_at	timestamptz null	read_at for in-app

notification_preferences Per user

Column	Type	Notes
user_id	FK PK	
channels	jsonb	per severity, e.g. critical: [app, sms]
quiet_hours	jsonb null	except critical

3.8 Audit, exports and device events

audit_log Who changed what (append-only)

Column	Type	Notes
id	bigserial	
tenant_id	varchar null	null for platform actions
actor_type, actor_id	enum, uuid	user platform_admin device system
action	text	user.invite, meter.update, flow.deploy, tenant.suspend ...
entity, entity_id	text	
before, after	jsonb	changed fields only; secrets removed
ip	inet	
at	timestamptz	

export_jobs Large CSV / PDF / Excel exports run in the background

Column	Type	Notes
id	uuid PK	
tenant_id, user_id	FK	
kind	enum	energy alerts readings users shift_energy
format	enum	csv pdf xlsx
params	jsonb	{from, to, meters, columns}
status	enum	queued running done failed
file_key	text null	object-storage key
expires_at	timestamptz	files deleted after 7 days

device_messages Telemetry log — every message per meter and polling cycle (hypertable, kept 7 days)

Column	Type	Notes
time	timestamptz	read time (device clock)
tenant_id, device_id	FK	
meter_id	FK null	null when the slave is unknown
slave	smallint	Modbus address
status	enum	ok late timeout crc out_of_range
latency_ms	integer null	received_at – time; null for timeouts
payload	jsonb	the reading exactly as sent (Raw button)

Column	Type	Notes
received_at	timestampz	

device_commands Commands sent to a device (Commands tab)

Column	Type	Notes
id	uuid PK	
code	varchar(12)	CMD-3FT7
tenant_id, device_id	FK	
command	enum	ping get_info sync_time set_polling read_register clear_buffer restart
params	jsonb	{slave, fc, address, count} / {interval}
sent_by	FK users	needs devices.command permission
status	enum	queued sent success failed timeout (10 s without a reply)
reply	jsonb null	the device's answer
reply_ms	integer null	
created_at, sent_at, replied_at	timestampz	

Commands travel to the device on `ed/{orgId}/{deviceCode}/cmd` (MQTT) or are picked up on the next HTTPS heartbeat; polled devices are handled by the Modbus poller directly. The reply comes back on `.../cmd/reply` with the command id. Restart and Clear stored readings require confirmation and are written to the audit log; register reads are read-only (no Modbus writes from the app).

publish_destinations Where a device's readings are forwarded (Data publish tab)

Column	Type	Notes
id	uuid PK	
tenant_id, device_id	FK	
name	text	unique per device
type	enum	mqtt https
url	text	mqtt://host:8883 or https://...
topic	text null	MQTT topic template with {org}, {device}, {meter}; no + or #
qos, retain	smallint, boolean	MQTT only
method	enum null	POST PUT (HTTPS)
auth_secret	text null	encrypted; only a masked hint (Bearer •••• 7f2a) is ever returned
interval	text	reading 60 300 900 seconds
format	enum	flat (one row per meter) grouped (one message per device)
fields	text[]	kw, kva, pf, v, i, hz, thd_v, thd_i, kwh
meter_ids	uuid[]	empty = all meters of the device
enabled	boolean	

publish_attempts Every publish (Data logs tab, together with device_commands) — hypertable, kept 30 days

Column	Type	Notes
time	timestampz	
destination_id	FK	
is_test	boolean	Test publish button
status	enum	success retried (delivered after a retry) failed (gave up after 3 tries)
detail	text	HTTP 200 / PUBACK · QoS 1 / HTTP 503 ...

Column	Type	Notes
duration_ms, bytes	integer	
payload	jsonb	the message sent or attempted

Publisher worker: a BullMQ repeatable job per enabled destination builds the payload from the live cache at each interval and sends it (MQTT client per broker, HTTPS with a 10 s timeout); failures retry twice with back-off, then the attempt is recorded as failed and, after 3 failures in a row, a 'Publish failing' alert is raised for the device.

device_events Online/offline changes, firmware updates, reboots

Column	Type	Notes
id	bigserial	
device_id	FK	
type	text	online offline weak_signal firmware_update config
detail	jsonb	
at	timestampz	

4. Page-by-page field reference

Every form field and table column on every page, with the API field / database column it maps to, its type and validation, and — where the current page is inconsistent or would lose data — the correction to make in the front end. The list was taken from the running pages (including forms built by scripts). * = required. Validation is enforced by the server; the page repeats it only for quick feedback.

4.1 Corrections summary

#	Where	Problem today	Correct field
1	Register, Accept invite, Change password	Three different password rules (letter + number; upper + lower + number)	One rule everywhere: at least 8 characters with at least one letter and one number; server also rejects common passwords
2	Register vs. other forms	Mobile is 10 digits (maxlength 10) on Register but +91 / 10–18 characters elsewhere	Accept a 10-digit Indian number or +country format; store E.164 (+919845012345)
3	Register	One 'Full name' field, but users have first and last name	Split into First name * and Last name (or the server splits at the first space)
4	Meter, sub-meter, device forms	Meter ID / Sub-meter ID / Device ID are editable text	Generated by the server (MTR-1013, SM-1001-05, GW-07); read-only in Edit
5	Edit meter, Edit sub-meter, Dashboard edit	Status offers Online / Idle — these come from readings	Replace with Mode: Active Maintenance Disabled; Online / Idle / Offline stay derived
6	Add / Edit meter, Dashboard edit	Category lists differ (6, 4 and 5 options); 'Generation' duplicates Meter type	One category list from the server: Incomer, HVAC, Lighting, Process, Utility, Other; generator is Meter type
7	Add meter vs. Add sub-meter	CT ratio is free text on meters, a fixed list on sub-meters; Edit lists are shorter than Add	Same Connection control everywhere: CT (primary / secondary A) or Direct (rating A)
8	Add / Edit sub-meter	No device or slave ID — but a sub-meter can be read by another device (Connections page)	Add 'Read by device' (default: parent's device) and 'Modbus slave ID'; keep register offset for multi-channel meters
9	All numeric inputs (rated load, contract demand, CO2 factor, grace period, allocation ...)	Plain text inputs	type=number with unit and range (see each field)
10	Alerts → Edit alert rule	A short form (metric, condition, threshold) — saving it would erase expression, windows and notification settings	Open the same full rule form used for New alert rule

#	Where	Problem today	Correct field
11	Alerts → Update alert	Status 'Open' here, 'Active' elsewhere; severity editable	Status: Open Acknowledged Resolved everywhere; severity read-only (comes from the rule); note required to resolve
12	Settings → Tariff slabs	Status Current / Expired is chosen by hand; Days is a 3-option list; no end date	Add Effective to; status is derived from the dates; Days as weekday checkboxes
13	Settings → Sites	Default tariff only on Add site; Flat has no rate	Default tariff on Add and Edit; Flat rate (₹/kWh) shown when Flat is chosen
14	Shifts	Create has Colour but no Status; Edit has Status but no Colour or 'None' break; supervisor lists differ	Same fields in Create and Edit; supervisors loaded from users with the Supervisor role
15	Roles	Edit role shows Type as editable text; permission key typed by hand	Type read-only (Built-in / Custom); permission key built from Module + Action lists
16	Users → User details	Shows the invitation link again	Tokens are stored hashed, so the link is shown once; offer Resend invite (new link) instead
17	Forgot password	New password can be set right after entering Org ID and email	Require the 6-digit code sent to the email before the new password is accepted; sign out all sessions afterwards
18	Settings Industry vs. Register Facility type	Two different lists	One list: Manufacturing plant, Commercial building, Education campus, Hospital, Data centre, Retail / mall, Solar / renewable plant, Other
19	My profile → Time zone	Only two time zones	Full IANA list (default Asia/Kolkata)

4.2 Every app page — top bar

My profile (dialog) · PATCH /me

Field	API field → column	Type · validation	Correction
First name *	firstName → users.first_name	text 1–50	—
Last name	lastName → users.last_name	text 0–50	—
Email *	email → users.email	email, unique in tenant	changing it sends a 6-digit code to the new address before it takes effect
Phone	mobile → users.mobile	E.164	see correction 2
Language	language → users.language	en-IN hi ta	—
Time zone	timezone → users.timezone	IANA name	full list (correction 19)

Change password (dialog) · POST /me/password

Field	API field → column	Type · validation	Correction
Current password *	currentPassword	checked against password_hash	—
New password *	newPassword → users.password_hash	8–72 chars, ≥1 letter and ≥1 number; ≠ current	one rule (correction 1); other sessions signed out
Confirm new password *	(not sent)	must equal New password	—

Export (dialog) · GET export data or POST /exports

Field	API field → column	Type · validation	Correction
From *	from	date	—
To *	to	date ≥ From; range ≤ 366 days	—
Format	format	csv pdf (xlsx later)	ranges over 31 days run as a background job

4.3 Sign-in and registration pages

login.html · POST /auth/login

Field	API field → column	Type · validation	Correction
Organisation ID	orgId → tenants.id	2–12 chars A–Z 0–9, upper-cased	—
Username	login	username, email or mobile	—
Password	password	1–72 chars	—
Remember me	remember → auth_sessions.remember	boolean: 30 days / 12 hours	—

admin-login.html · POST /admin/auth/login

Field	API field → column	Type · validation	Correction
Username	username → platform_admins.username	text	—
Password	password	text	add a TOTP code step (MFA)

register.html · POST /auth/register

Field	API field → column	Type · validation	Correction
Organisation name *	orgName → tenants.name	text 2–120	—
Facility type *	industry → tenants.industry	enum (one list)	correction 18
Site location *	city → tenants.city, first sites row	text 2–80	—
Electricity provider (DISCOM)	discom → tenants.discom, sites.utility	text 0–40	—
Contract demand	contractDemandKva → tenants.contract_demand_kva	integer 1–100000 kVA	number input with kVA unit
Number of energy meters *	meterRange → tenants.meter_range	1-10 11-50 51-200 200+	—
Full name *	firstName, lastName → users	text	correction 3
Work email *	email → users.email, tenants.contact_email	email	—
Mobile number *	mobile → users.mobile, tenants.phone	E.164	correction 2
Username *	username → users.username	3–30 chars a–z 0–9 . _ ; unique platform-wide	live check GET /auth/username-available
Password *	password → users.password_hash	8–72 chars, ≥1 letter and ≥1 number	correction 1
Confirm password *	(not sent)	equals Password	—

verify-email.html · POST /auth/verify-email

Field	API field → column	Type · validation	Correction
6-digit code	code → otp_codes (verify_email)	6 digits; 10 min; 5 tries	—

accept-invite.html · POST /invites/:token/accept

Field	API field → column	Type · validation	Correction
First name *	firstName → users.first_name	text 1–50	—
Last name	lastName → users.last_name	text 0–50	—
Username	username → users.username	unique platform-wide	—

Field	API field → column	Type · validation	Correction
Mobile	mobile → users.mobile	E.164	—
Create password *	password → users.password_hash	8–72 chars, ≥1 letter and ≥1 number	—

forgot-password.html · POST /auth/forgot-password → /auth/reset-password

Field	API field → column	Type · validation	Correction
Organisation ID	orgId	as login	—
Work email	email	email	always answer 'if the account exists, a code was sent'
(new) Code	code → otp_codes (reset_password)	6 digits	correction 17 — add this step
New password / Confirm	password	8–72 chars, ≥1 letter and ≥1 number	—

login-approval.html / awaiting-approval.html

Field	API field → column	Type · validation	Correction
Organisation ID, Username, Password	POST /auth/login	returns TENANT_PENDING / TENANT_SUSPENDED with reason and date	—

4.4 Dashboard

Edit meter (dialog) · PATCH /meters/:id

Field	API field → column	Type · validation	Correction
Meter name	name → meters.name	text 2–80	—
Meter ID	code → meters.code	read-only	correction 4
Location	location → meters.location	text 0–80	—
Category	category → meters.category	enum	correction 6
Rated load	ratedKw → meters.rated_kw	number > 0, kW	correction 9
Status	mode → meters.mode	active maintenance disabled	correction 5

Energy Meters table · GET /meters?view=tree + live

Column	Source
Meter	meters.name, code, phase tag
Location	meters.location
Sub-meters	count of children
Power	live kW (WebSocket)
Energy (kWh)	today's kWh from readings_15m
Load	kW / rated_kw
Status	derived status

Summary tiles · GET /dashboard/summary

Column	Source
Live load, Today's energy, Cost, CO2, Max demand, Peak	analytics (section 8 formulas)

4.5 Energy Meters (meters.html)

Add meter (dialog) · POST /meters

Field	API field → column	Type · validation	Correction
Meter name *	name → meters.name	text 2–80	—
Meter ID	code	generated	correction 4
Category	category	enum	correction 6
Supply	phase → meters.phase (+ nominal_v)	3 (415 V) 1 (230 V)	—
Make & model *	meterModelId → meters.meter_model_id	from /catalogue/meter-models	sets meter class, register map and allowed wiring
Serial number	serialNo → meters.serial_no	text 0–40	—
Meter class *	meterClass → meters.meter_class	basic multi pq dc dg	required; Create is blocked until chosen
Site	siteId → meters.site_id	from /sites	—
Location / panel	location	text 0–80	—
Reports via device *	deviceId → meters.device_id	from /devices	—
Modbus slave ID	slaveId → meters.modbus_slave_id	integer 1–247; unique on the device	required when a device is chosen
Register map	registerMapId	set by the model; Custom = upload list	—
Meter type	kind → meters.kind	consumer generator	Generator suggests Outgoing
Incoming / Outgoing *	energyDirection → meters.energy_direction	import export	required; Create is blocked until chosen
Rated load	ratedKw → meters.rated_kw	number > 0, kW	correction 9
CT primary / CT secondary	ctPrimary, ctSecondary / directA	5–10000 A / 5 A 1 A Direct connected	correction 7
Wiring	wiring	3P4W 3P3W 1P2W (model's list only)	—
PT ratio	ptRatio	e.g. 11000/110; empty = LT	—
Commissioned on / Calibration due	commissionedOn, calibrationDue	dates; due ≥ commissioned	—

Edit meter (dialog) · PATCH /meters/:id

Field	API field → column	Type · validation	Correction
Same fields as Add meter			Category list identical to Add (correction 6)
Status	mode → meters.mode	active maintenance disabled	correction 5
Reports via device *	deviceId		changing it re-routes live readings; confirm first

Add sub-meter (dialog) · POST /meters/:id/sub-meters

Field	API field → column	Type · validation	Correction
Parent meter *	parentId → meters.parent_id	a main meter	—
Sub-meter name *	name	text 2–80	—
Sub-meter ID	code	generated SM-####-##	correction 4
Connection	ctPrimary, ctSecondary / directA	as meter	correction 7
Supply	phase	1 3; a 3φ sub-meter needs a 3φ parent	—

Field	API field → column	Type · validation	Correction
Meter class *	meterClass	basic multi pq dc dg	required
Incoming / Outgoing *	energyDirection	import export	required
Rated load	ratedKw	number > 0	—
Cost centre	costCentreId → meters.cost_centre_id	from /cost-centres	—
Modbus register offset	registerOffset	integer 0–65535	—
(new) Read by device	deviceId	default parent's device	correction 8
(new) Modbus slave ID	slaveId	1–247	correction 8

Edit sub-meter (dialog) · PATCH /meters/:id

Field	API field → column	Type · validation	Correction
Same fields as Add sub-meter			same option lists as Add (correction 7)
Status	mode	active maintenance disabled	correction 5

Meter tree table

Column	Source
Meter / Sub-meter	name, code, phase tag, model
Location	location
Device	device code + name, status dot
Type	sub-meter count or connection (CT 400/5)
Power	live kW; main meters also Σ sub-meters
Energy (kWh)	today; Δ unmetered %
Load	kW / rated_kw
Status	derived

4.6 Meter page (meter.html)

Panels · GET /meters/:id, /live, /readings, /summary

Column	Source
Header	name, code, model, phase tag, device, status
Gauges	load %, PF, voltage (avg L-N or L-N), voltage THD (limit 8 % · IEEE 519), current (max phase or single), current THD, Hz — each with its name, unit and status above the dial
Header chips	phase tag; ↓ Incoming · Import or ↑ Outgoing · Export (energy_direction)
Electrical parameters (Phase Voltage Current Power factor kVA kW VTHD ITHD)	per phase L1–L3 (or L–N); Total row = meter totals with PF Lead/Lag; L–L row = line voltage + imbalance; list below repeats PF, kVA, kW, VTHD, ITHD and adds kVAR, Hz, energy import / export registers, rated load
Trends tab: kW and kVA · Power factor (target 0.95) · Voltage harmonics L1–L3 · Current harmonics L1–L3 (limit 8 %)	GET /meters/:id/trends?params=kw,kva,pf,thd_v1..3,thd_i1..3 → readings_15m; View as table
Import / export energy (Today This month Meter register)	readings_1d kwh / kwh_exp and kwh_imp / kwh_exp registers; the direction not measured shows Not measured
Meter specification	meter_models (accuracy, comms) + meters (serial_no, wiring, CT, PT, slave on device, serial line, commissioned_on, calibration_due)
Modbus register map (Parameter Register Data type Unit)	register_maps.registers for the model; message to load the maker's list when none

Column	Source
Power today (Time kW)	readings_15m kw_avg
Daily energy (Day kWh)	readings_1d kwh
Sub-meters (Sub-meter CT Power Energy Status)	children with live values

New / Edit alert rule (dialog) · POST /alert-rules, PUT /alert-rules/:id

Field	API field → column	Type · validation	Correction
Rule name *	name → alert_rules.name	text 1–80	—
Severity	severity	critical warning info	—
Meter * (when opened from Alerts)	meter → meter_id	a meter or sub-meter	—
Applies to	scope, subs → scope, sub_ids	meter subs meter_and_subs	—
Check	evaluate	live 15m 1h day	—
Conditions (guided)	cond.rows[{param, op, v1, v2}], cond.match	param per meter phase; v2 for between/outside	1φ meters: no imbalance
Expression	cond.expr → compiled	whitelisted variables only	validated on save
For at least	cond.hold	0 1 5 15 30 60 min	—
With the latest readings / when	active.mode, shifts, slots	always shifts slots custom	—
From / To / Days	active.from, to, days	HH:MM (may cross midnight); ≥1 day	—
Send by	notify.channels	app email sms whatsapp webhook; ≥1	—
To (roles)	notify.roles	role names	—
Specific users	notify.users	user ids	—
Also email	notify.emails	valid emails, max 10	—
While it stays true	notify.repeat	once 15m 1h	—
Close the alert	notify.resolve	auto5 auto15 manual	—
If nobody acknowledges it	notify.escalate	0 15 30 60 min	—

4.7 Device Management (devices.html, device.html)

Add / Edit device (dialog) · POST /devices, PATCH /devices/:id

Field	API field → column	Type · validation	Correction
Device name *	name → devices.name	text 2–80	—
Device ID	code	generated GW-##	correction 4
Device type	type	modbus_tcp_gateway energy_server edge_nodered edge_4g lorawan_gateway	filled from the model
Make & model *	deviceModelId → devices.device_model_id	from /catalogue/device-models	fills type, field protocol, uplink, default port; shows ports and power
Serial number	serial	text; unique in tenant	—
Site	siteId	from /sites	—
Installed at	location	text 0–80	—
Connectivity	link	ethernet wifi lte	—
IP address	ip	IPv4 / IPv6; required for polled uplink	—

Field	API field → column	Type · validation	Correction
Port	port	integer 1–65535 (502 Modbus, 8883 MQTT)	number input
Field protocol	fieldProtocol	Modbus RTU (RS-485) Modbus TCP LoRaWAN	—
Uplink	uplink	modbus_tcp_poll mqtt_push https_push	—
Polling interval	pollingS	15 30 60 300	—
RS-485 baud rate	baud	4800 9600 19200 38400 57600 115200	only for a serial field bus
Parity	parity	none even odd	—
Data bits · stop bits	dataBits, stopBits	8-1 8-2 7-1	must match every meter on the line
Reply timeout	timeoutMs	integer 100–10000 ms	—
Retries	retries	0–3	—
Time sync	timeSync	ntp server	—
Client ID (read-only)	device_credentials.client_id	{orgId}.{Device ID}	updates as Device ID changes
Device token * (read-only)	returned by POST /devices	edt_ + 40 base-62 chars; Copy and Generate again buttons	shown once; stored as SHA-256 hash
I've copied the token ...	(not sent)	must be ticked to create	—
Edit: Device token	token_last4 · created_at	masked edt_****a91F	—
Edit: Regenerate	POST /devices/:id/token/regenerate	confirm first; new token shown once	device offline until updated

Device page → Telemetry log · GET /devices/:id/telemetry + WebSocket 'reading'

Column	Source
Time, Meter (slave, ID, phase, meter class)	device_messages.time, slave → meters
Value columns follow the meter classes in view: kW, Voltage, Current, PF, Hz, V-THD, I-THD, Fuel, Run hours, kWh register	payload; n/a where a class has no such field
Telemetry chart tabs	the chartable fields of the meters in view + Latency
Latency, Status	latency_ms, status (OK, Late, Timeout, CRC error, Out of range)
Filters: meter, status (incl. problems only), last 15 min / 1 h / 6 h; Live switch; Raw; Export CSV	query parameters / live events
Summary chips: messages, % OK, lost, average latency	computed for the filtered view

Device Management → Telemetry logs (all devices) · GET /telemetry + WebSocket 'reading'

Column	Source
Time, Device (polling interval), Meter (slave, ID, phase, class), class-dependent values, Latency, Status, Raw	device_messages across the tenant's devices
Filters: device, meter (narrows to the device), status (incl. problems only), last 15 min / 1 h / 6 h; Live switch; Show more (100 rows); Export CSV	query parameters, paged
Summary chips: messages, % OK, lost, average latency	computed for the filtered view

Device page → Network & protocol / Hardware & firmware

Column	Source
Serial line (e.g. RS-485 2-wire · 9600 8E1), Modbus timing (timeout, retries, polling), Uplink endpoint (MQTT topic + client ID / HTTPS ingest / polled IP:502)	devices
Manufacturer, Ethernet, serial ports, cellular / wireless, I/O, protocols, power supply, field nodes	device_models

Device page → Commands · POST /devices/:id/commands

Field	API field → column	Type · validation	Correction
Command	command	ping get_info sync_time set_polling read_register clear_buffer restart	restart / clear ask for confirmation
Meter (register read)	params.slave	a slave of this device	—
Function	params.fc	FC03 FC04	read only
Start address / Registers	params.address, params.count	0–65535 / 1–32, whole numbers	—
Interval (set polling)	params.interval	15 s 30 s 1 min 5 min	—

Device page → Data publish · Add / Edit destination (dialog)

Field	API field → column	Type · validation	Correction
Name *	name	1–60, unique per device	—
Type	type	mqtt https	—
Publish	interval	every reading 1 5 15 min	—
Broker / Webhook URL *	url	mqtt://host:port https://...	—
Topic * (MQTT)	topic	template, no + or #	—
QoS, Retain (MQTT)	qos, retain	0–2, boolean	—
Method, Authorization header (HTTPS)	method, auth_secret	POST PUT; secret write-only	shown masked; empty keeps the old one
Payload	format	flat grouped	—
Fields	fields	≥ 1	—
Meters	meter_ids	≥ 1 (all = empty)	—
Publish now	enabled	boolean	—

Device page → Data logs · GET /devices/:id/data-logs

Column	Source
Time, Type (Publish / Command), Destination / command, Detail, Status, Duration, Size, View	publish_attempts + device_commands
Filters: type, status, last 1 / 6 / 24 h, search; Export CSV; chips: entries, % delivered, failed	query parameters

Delete device? (dialog) · DELETE /devices/:id?moveTo=

Field	API field → column	Type · validation	Correction
Move its meters to *	moveTo → meters.device_id	another device; required if it has meters	slave-ID clashes reported before deleting

Devices table / Connected meters table

Column	Source
Device / Meter	code, name, type
Location	location, site
Meters	count; slave IDs
Network	ip;port, link
Signal	signal_pct
Firmware	firmware, update badge if firmware_available
Status	derived, last_seen
Address / Polling (device page)	modbus_slave_id, polling_s

4.8 Connections (connections.html)

Canvas · GET / PUT /connections/flow, POST /deploy

Field	API field → column	Type · validation	Correction
Blocks	nodes[<code>{id, type, name, ref, x, y, props}</code>]	types: submeter meter virtual device	—
Wires	wires[<code>{from, to, kind}</code>]	direct sub link; rules in section 3.4	validated again on the server before deploy

4.9 Analytics (analytics.html)

Edit target & allocation (dialog) · PUT /energy-targets/:meterId/:month

Field	API field → column	Type · validation	Correction
Monthly target	targetKwh → energy_targets.target_kwh	number > 0, kWh	number input
Cost centre	costCentreId	from /cost-centres	—
Allocation	allocationPct	0–100 %	number input
Tariff	tariffMode	tod flat	—
Notes	notes	text 0–500	—

Sub-meter consumption report · GET /analytics/sub-meter-report

Column	Source
Sub-meter	name, code
Cost centre	cost_centres.code
Energy (kWh), Cost	readings_1d + ToD cost
Target (kWh), vs target	energy_targets, % difference
Avg load	average kW / rated_kw

4.10 Alerts (alerts.html)

Update alert (dialog) · PATCH /alerts/:id

Field	API field → column	Type · validation	Correction
Status	status → alerts.status	open acknowledged resolved	correction 11
Severity	(read-only)	from rule	correction 11

Field	API field → column	Type · validation	Correction
Assign to	assignedTo → alerts.assigned_to	active user	—
Resolution note	note → alerts.note	text 0–1000; required to resolve	—

Edit alert rule (short dialog)

Field	API field → column	Type · validation	Correction
Metric, Condition, Threshold, For at least, Severity, Active during, Applies to, Notify			correction 10 — use the full rule form

New alert rule (dialog)

Field	API field → column	Type · validation	Correction
Same fields as the meter page		+ Applies to: all all_meters all_subs for tenant-wide rules	—

Alert log / Alert rules tables · GET /alerts, /alert-rules

Column	Source
Alert	title, severity, category
Source	meter path + code, or device
Value / limit	value_text / limit_text
Raised	raised_at
Status	open / acknowledged / resolved
Rule, Metric, Condition, Applies to, Notify via, Severity, Enabled	alert_rules fields (summary text built by the front end)

4.11 Shift management (shifts, schedule, instances)

Create / Edit shift (dialog) · POST /shifts, PATCH /shifts/:id

Field	API field → column	Type · validation	Correction
Shift name *	name → shifts.name	text 2–40	—
Code	code	unique, e.g. SH-D, upper-case	required
Colour	color	amber orange violet cyan green	also in Edit (correction 14)
Status	status	active inactive	also in Create (correction 14)
Start time / End time	startTime, endTime	HH:MM; not equal	—
An overnight shift counts on	countOn	start_day end_day	shown only when End < Start
Break	breakMin	0 30 45 60	'None' also in Edit
Supervisor	supervisorId	user with Supervisor role	correction 14
Grace period	graceMin	integer 0–60 min	number input
Energy target	energyTargetKwh	number ≥ 0, kWh per run	—

New / Edit schedule (dialog) · POST /shift-schedules

Field	API field → column	Type · validation	Correction
Name *	name → shift_schedules.name	text 2–60	—
Shift *	shiftId → shift_schedules.shift_id	from /shifts	—

Field	API field → column	Type · validation	Correction
Repeats on	days	bit mask, ≥ 1 day	—
Effective from	startsOn	date	—
Until	endsOn	date ≥ Effective from, or empty	—

Assign staff to shift (dialog) · POST /shift-assignments

Field	API field → column	Type · validation	Correction
Employees	userIds → shift_assignments.user_id (one row each)	active users	—
Shift	shiftId		—
From / To	fromDate, toDate	To ≥ From; no overlap per user	—

Shifts, Schedules, Shift instances tables

Column	Source
Shift, Timing, Duration, Supervisor, Staff, Status	shifts + count of assignments
Midnight crossover shifts	derived from start/end and count_on
Schedule, Shift, Repeats, Effective, Staff, Status	shift_schedules
Instance, Planned / actual, Staff, Energy kWh, kWh / unit, Status	shift_instances (kWh / unit = energy_kwh / production_units)
Midnight crossover runs (kWh on start day / next day)	energy split at 00:00 from readings_15m

4.12 Users (users.html)

Invite user (dialog) · POST /users

Field	API field → column	Type · validation	Correction
First name *	firstName	text 1–50	—
Last name	lastName	text 0–50	—
Work email *	email	email, unique in tenant	—
Mobile	mobile	E.164	—
Employee ID *	empId → users.emp_id	^[A-Za-z]{2,5}-?[0-9]{1,6}\$, unique in tenant; default next	—
Department	department	from tenant list	—
Role *	roleId	from /roles	—
Shift	shiftId	from /shifts	—
Site access	siteIds → user_sites	none = all sites	—

Edit user (dialog) · PATCH /users/:id

Field	API field → column	Type · validation	Correction
Same fields as Invite			—
Status	status	active inactive suspended	'invited' cannot be chosen; it ends when the invite is accepted

Invitation created / User details

Field	API field → column	Type · validation	Correction
Invitation link	returned once by POST /users		correction 16

Users table · GET /users?page&size;

Column	Source
User	name, username, email (masked by field permissions)
Role, Shift, Department	users
Status	users.status
Last active	users.last_active_at

4.13 Roles & Permissions (roles.html)

Create / Edit role (dialog) · POST /roles, PATCH /roles/:id

Field	API field → column	Type · validation	Correction
Role name *	name → roles.name	text 2–40, unique in tenant	—
Copy permissions from	copyFrom	role id (not stored)	Create only
Type	(read-only) is_system	Built-in Custom	correction 15
Default landing page	landingPage	dashboard meters analytics alerts	same list in Create and Edit
Description	description	text 0–200	—
Permission matrix	permissions[] → role_permissions	permission keys	—

Add / Edit permission (dialog) · POST /permissions

Field	API field → column	Type · validation	Correction
Permission key *	key → permissions.key	module.action	built from Module + Action (correction 15)
Module	module	enum	—
Description	description	text 0–200	—
Grant to roles	roleIds → role_permissions		—

All roles / Permissions tables

Column	Source
Role, Type, Users, Permissions, Created	roles + counts
Key, Module, Description, Roles using	permissions + count

4.14 Settings (settings.html)

Organisation profile · PATCH /tenant

Field	API field → column	Type · validation	Correction
Organisation ID	tenants.id	read-only	—
Workspace URL *	slug	^[a-z0-9][a-z0-9-]{1,29}\$, unique	—
Legal name *	name	text 2–120	—
GSTIN	gstn	15-char GSTIN pattern, upper-cased	—

Field	API field → column	Type · validation	Correction
Industry	industry	enum	correction 18
Contact email	contactEmail	email	—
Time zone	timezone	IANA	—
Currency	currency	INR USD AED	—
Energy unit	energyUnit	kWh MWh	—
CO2 emission factor	co2Factor	number 0–2 kg/kWh	number input
Registered address	address	text 0–300	—

Add / Edit site (dialog) · POST /sites, PATCH /sites/:id

Field	API field → column	Type · validation	Correction
Site name *	name	text 2–80	—
Address	address	text 0–200	—
Utility / DISCOM	utility	text 0–40	—
Consumer number	consumerNo	text 0–30	—
Contract demand	contractDemandKva	integer ≥ 0 kVA	number input
Default tariff	defaultTariff (+ flatRate)	tod flat	correction 13
Status (Edit)	status	active inactive	—

Energy cost → Instances (tariff-instances.html) · GET /tariff-instances

Column	Source
Running now, Energy cost, Energy & average ₹/kWh, Peak-slot share	summary for the period
By slot: Slot, Tariff, Runs, Energy, Rate, Cost, Share of cost, Avg kW	tariff_instances grouped by slot
Instance, Slot, Date, Time (+1 when it crosses midnight), Status, Energy, Rate, Cost, Peak kW	tariff_instances
Filters: period (today, 7, 14, 30 days), slot, status, search	query parameters

Energy cost → Tariff · Add / Edit tariff slab (dialog) · POST /tariffs, PATCH /tariffs/:id — moved here from Settings

Field	API field → column	Type · validation	Correction
Time slot *	slot	T1–T5	—
Name *	name	text 2–40	—
From / To	startTime, endTime	HH:MM; may cross midnight	no overlap with other slots on the same days
Energy rate	energyRate	number > 0 ₹/kWh	—
Demand charge	demandCharge	number ≥ 0 ₹/kVA	—
Days	days	weekday bit mask	correction 12
Effective from	validFrom	date	—
(new) Effective to	validTo	date ≥ from, or empty	correction 12
Status (Edit)	(derived)	current expired	correction 12

Session details (dialog) · PATCH /auth/sessions/:id

Field	API field → column	Type · validation	Correction
Device label	deviceLabel → auth_sessions.device_label	text 0–60	—

Sites / Tariff slabs / Your active sessions tables

Column	Source
Site, Utility, Contract demand, Meters, Status	sites + meter count
Slot, Tariff, Time, Days, Energy rate, Demand charge, Status	tariffs (status derived)
Device, Location, IP, Last active	auth_sessions (location from IP lookup); Sign out = DELETE

4.15 Control Center (tenants, approvals, tenant page)

Suspend temporarily (dialog) · POST /admin/tenants/:id/suspend

Field	API field → column	Type · validation	Correction
Suspend for	(preset)	1 day 3 days 1 week 30 days date	fills Suspended until
Suspended until *	until → tenants.suspended_until	datetime in the future	—
Reason	reason → tenants.status_reason	text 0–120; shown to the tenant	—

Limits (table with inputs) · PUT /admin/tenants/:id/limits

Field	API field → column	Type · validation	Correction
Limit	value → tenant_limits.value	integer ≥ used, or 0	lowering below usage is refused
Unlimited	value = 0	checkbox	—
(reason)	reason	text 0–200	ask for a reason when saving

Tenants / Pending requests / Users tables

Column	Source
Organisation, Status, Site · provider, Meters, Contract demand, Users	tenants + usage counts
Facility, Requested	industry, created_at
User, Role, Department, Status, Last active	users (read-only)

5. Services (backend modules)

Each service below is one NestJS module with a controller (HTTP), a service (business rules) and a repository (database). The ingestion, alert and job workers run as separate processes from the API so heavy traffic never slows down page loads.

AuthService — Sign-in, sessions and one-time codes

Responsibilities	• Sign in with Org ID + username / email / mobile + password; checks tenant status (pending → awaiting approval page, suspended → reason and date) and user status. • Optional 2FA: password then 6-digit OTP; OTP-only sign-in by mobile/email. • Issues a 15-minute access JWT and a rotating refresh token in an httpOnly cookie; remember-me 30 days, otherwise 12 hours. • Forgot / reset password, verify email, login approval, switch tenant (same email in another active tenant), list and revoke sessions.
Main methods	login, refresh, logout, me, requestOtp, verifyOtp, forgotPassword, resetPassword, switchTenant, listSessions, revokeSession

PlatformService (Control Center) — Tenant approvals, suspension and limits

Responsibilities	• Separate realm and 8-hour session for platform admins, MFA required. • Approve or reject pending registrations (emails the tenant admin), suspend with reason and optional end date, reactivate, delete. • Revokes every session of a suspended tenant immediately; scheduler lifts timed suspensions. • Edit limits per tenant with a reason; view a tenant's users and usage.
Main methods	listTenants, getTenant, approve, reject, suspend, reactivate, setLimits, deleteTenant

TenantService — Organisation profile, registration and limits

Responsibilities	• Self-registration creates a pending tenant + its admin user (generates an 8-character Org ID or reserves the chosen slug). • Profile settings (name, GSTIN, address, utility, contract demand, time zone, CO2 factor). • LimitService.check(tenant, key) is called inside the same transaction as every create — the server, not the browser, enforces limits (409 LIMIT_REACHED).
Main methods	register, getProfile, updateProfile, getLimits, usage, check

UserService + InviteService — Team members

Responsibilities	• List with search, role, status, department filters and server-side paging. • Invite (creates user in 'invited' state, emails a link), resend, accept (sets password), suspend, deactivate, delete. • Next employee ID, email unique per tenant, username unique platform-wide, site restrictions.
Main methods	list, get, invite, update, remove, resendInvite, acceptInvite, nextEmpId

RoleService + PermissionService — Roles, module permissions and field permissions

Responsibilities	• CRUD custom roles; built-in roles are seeded per tenant and cannot be deleted. • Permission matrix per module/action; field permissions per role (view/edit). • A guard checks module permissions on every route; a response interceptor removes fields the role may not view and rejects writes to fields it may not edit.
Main methods	listRoles, createRole, updateRole, deleteRole, getMatrix, setMatrix, getFieldPerms, setFieldPerms, can, canField

SiteService — Locations

Responsibilities	• CRUD sites with utility, consumer number and contract demand; limit 'sites'.
Main methods	list, create, update, remove

DeviceService — Gateways and loggers

Responsibilities	• CRUD devices; generates the device token on create and returns it once; regenerate token (old one revoked, device disconnected). • Lists the meters a device reads (with slave IDs); firmware info and update requests; status history. • Stores the serial line (standard, baud, parity, data / stop bits) and Modbus timeout / retries; checks the uplink is one the model supports. • Hidden fields (IP, MAC, serial) filtered by field permissions.
Main methods	list, get, create (→ device + token), update, remove, meters, regenerateToken, requestFirmwareUpdate

CatalogueService — Hardware catalogue

Responsibilities	- Lists real device models and meter models with their published specifications; tenants may add their own entries. - Register maps per meter model (1-based addresses, function code, data type, word order, unit); import from the maker's CSV; mark as verified.
Main methods	deviceModels, meterModels, registerMap, importRegisterMap, verifyRegisterMap

MeterService — Meters, sub-meters and virtual meters

Responsibilities	- Meter tree (main meters with their sub-meters), filters by site, status, consumer/generator. - Create meter / sub-meter with Supply (1ϕ / 3ϕ), CT ratio, rated kW, device and slave ID; limits 'meters' and 'subMeters'. - Validates that (device, slave ID) is free and that 1ϕ meters have no v2/v3 parameters in their rules. - Requires meter class and Incoming / Outgoing on create; wiring limited to the model's list; keeps the commissioning record (serial no., CT / PT, dates). - Trends series (kW, kVA, PF, per-phase THD) from readings_15m; import / export energy summary. - Maintenance mode on/off; archive instead of hard delete.
Main methods	tree, get, create, createSub, update, setMaintenance, archive, trends, energySummary

ConnectionService — Connections canvas

Responsibilities	Load registry-based default flow, save draft, validate, deploy (versioned), list versions, roll back.
Main methods	getFlow, saveDraft, validate, deploy, versions, rollback

IngestionService (worker) — Getting readings in

Responsibilities	- MQTT consumer, HTTPS ingest endpoint and Modbus TCP poller all produce the same normalised reading. - Authenticates the device, maps slave ID → meter via a cached device map, applies scaling, rejects impossible values, flags register resets. - Deduplicates on (meter, time), stamps received_at, appends to Redis Stream 'readings', updates device last_seen.
Main methods	handleMqttMessage, handleHttpBatch, pollDevice, normalise, publish

TelemetryService — Storing and reading time-series

Responsibilities	- DB writer consumer: batches stream entries and inserts with COPY (every 1 s or 500 rows). - Query API for charts: picks raw or the right aggregate, fills gaps, returns only requested parameters. - Daily jobs: shift instance snapshots, daily energy totals.
Main methods	writeBatch, series, latest, energy, demand

LiveService + RealtimeGateway — Realtime to the browser

Responsibilities	- Consumes the stream, updates live:{meter} in Redis, computes load %, imbalance, Σ sub-meters and Δ unmetered, today's kWh and cost. - Emits to Socket.IO rooms, at most once per second per meter, filtered by field permissions (e.g. no cost for Operators).
Main methods	onReading, snapshot, emit

AnalyticsService — Reports and the Dashboard

Responsibilities	- Dashboard summary (live load, today's kWh, cost, CO₂, max demand vs contract, peak time). - Consumption by meter / sub-meter / site / shift / day, sub-meter report, power quality, solar vs forecast.
Main methods	dashboardSummary, consumption, subMeterReport, demand, powerQuality

TariffService + CostService — Money

Responsibilities	CRUD ToD slots; splits energy by slot (in the site's time zone) and applies energy rate and demand charge.
Main methods	list, create, update, remove, rateAt, cost

ShiftService — Shifts, schedules, instances

Responsibilities	• CRUD shifts and schedules; generates shift instances 14 days ahead on the days each schedule repeats; resolves overnight shifts and the 'count on' day; closes instances and stores their energy.
Main methods	shifts CRUD, schedules CRUD, generateInstances, closeInstance, instanceEnergy

AlertRuleService — Rule management

Responsibilities	• CRUD rules with the UI's JSON shape; validates parameters against the meter's phase; compiles expressions safely; limit 'alertRules'. • Dry-run: replays the last 24 hours and reports when the rule would have fired.
Main methods	list, create, update, remove, compile, test

AlertEngine (worker) — Evaluating rules 24×7

Responsibilities	• Live rules on every reading, window rules when a window closes, 'no readings' rules from the offline checker. • State machine with hold time, active windows, repeat, auto-resolve and escalation (section 7). Writes alerts and alert_events, queues notifications.
Main methods	onReading, onWindowClose, evaluate, transition

AlertService — The Alert log

Responsibilities	• List/filter/page alerts, acknowledge, acknowledge all, resolve with note, delete (admin).
Main methods	list, acknowledge, acknowledgeAll, resolve, remove

NotificationService (worker) — Delivery

Responsibilities	• Resolves roles → users, applies preferences and quiet hours, sends via app (WebSocket + inbox), email, SMS, WhatsApp or signed webhook; retries with back-off; records status.
Main methods	enqueue, send, markRead, inbox

ExportService (worker) — CSV / PDF / Excel

Responsibilities	• Small exports are generated on request; large ranges (over 31 days or 50k rows) become a background job with a download link. Columns match the current exports (Date, Meter ID, Meter, Parent meter, Location, Phase, Energy, Peak, Cost, CO2).
Main methods	create, status, download

SchedulerService — Timed jobs (BullMQ repeatable)

Responsibilities	• Every 30 s: device and meter offline check. Every minute: window-rule evaluation, timed-suspension lift, escalations. Hourly: invite/OTP clean-up. Nightly: shift instance generation, export clean-up, usage counters.
Main methods	register jobs

AuditService — Traceability

Responsibilities	• Interceptor records every create/update/delete with before/after (secrets removed); queryable by Tenant Admin and Auditor.
Main methods	record, list

6. REST API

Base URL `/api/v1`. JSON in and out. Authenticated with `Authorization: Bearer <access token>`; the refresh token travels only in an `httpOnly` cookie. The tenant always comes from the token, never from the URL, so one tenant can never ask for another's data.

6.1 Conventions

- **Paging:** `?page=1&size=10` → `{ data: [...], page, size, total }`; size 0 = all (capped at 1000). Matches the front end's "Showing 1–10 of 18" pager.
- **Filtering & sorting:** `?search=&status=&sort=-raised_at`.
- **Errors:** `{ error: { code, message, field? } }` with codes such as `VALIDATION`, `UNAUTHENTICATED`, `FORBIDDEN`, `NOT_FOUND`, `CONFLICT`, `LIMIT_REACHED` (409), `TENANT_PENDING`, `TENANT_SUSPENDED`, `OTP_LOCKED`, `RATE_LIMITED` (429).
- **Time:** ISO-8601 UTC in and out; `?tz=` optional for day/shift grouping (defaults to the tenant's time zone).
- **Idempotency:** POSTs that create things accept an `Idempotency-Key` header.
- **Docs:** OpenAPI generated from the code at `/api/docs`.

Auth

Method	Path	Purpose
POST	<code>/auth/register</code>	Register a tenant + admin (→ pending)
POST	<code>/auth/verify-email</code>	Confirm the 6-digit code
POST	<code>/auth/login</code>	{orgId, login, password, remember} → token, or {challenge:'otp'}
POST	<code>/auth/otp/request</code> · <code>/auth/otp/verify</code>	OTP sign-in / 2FA step
POST	<code>/auth/refresh</code>	Rotate tokens (cookie)
POST	<code>/auth/logout</code>	Revoke this session
POST	<code>/auth/forgot-password</code> · <code>/auth/reset-password</code>	Reset flow
GET	<code>/auth/me</code>	User, tenant, role permissions, field permissions, limits
GET	<code>/auth/memberships</code>	Other tenants with the same email
POST	<code>/auth/switch-tenant</code>	{orgId}
GET · DELETE	<code>/auth/sessions[:id]</code>	Active sessions / sign out a device
GET · POST	<code>/invites/:token[/accept]</code>	Show and accept an invitation

Control Center (platform realm)

Method	Path	Purpose
POST	<code>/admin/auth/login</code>	Platform admin sign-in (+ MFA)
GET	<code>/admin/tenants?status=</code>	Tenants and approvals
GET	<code>/admin/tenants/:id</code>	Profile, usage, limits, history
POST	<code>/admin/tenants/:id/approve</code> · <code>/reject</code> · <code>/suspend</code> · <code>/reactivate</code>	{reason, until}
PUT	<code>/admin/tenants/:id/limits</code>	{users: 60, devices: 0, reason}
GET	<code>/admin/tenants/:id/users</code>	Read-only list
DELETE	<code>/admin/tenants/:id</code>	Soft delete

Tenant, users, roles, sites

Method	Path	Purpose
GET · PATCH	/tenant	Organisation profile
GET	/tenant/limits	Limit, used and remaining per key
GET · POST	/users	List (filters, paging) / invite
GET · PATCH · DELETE	/users/:id	
POST	/users/:id/resend-invite · /suspend · /activate	
GET · POST	/roles	
PATCH · DELETE	/roles/:id	
GET · PUT	/roles/:id/permissions	Module × action matrix
GET · PUT	/roles/:id/field-permissions	module.field → {view, edit}
GET · POST · PATCH · DELETE	/sites[:id]	
GET · POST	/permissions	Permission key catalogue
GET · POST · PATCH · DELETE	/cost-centres[:id]	
PATCH · POST	/me · /me/password	My profile / Change password

Devices, meters, connections

Method	Path	Purpose
GET · POST	/devices	POST returns { device, token } — the token only this once
GET · PATCH · DELETE	/devices/:id	
GET	/devices/:id/meters	Meters it reads, with slave IDs
POST	/devices/:id/token/regenerate	New device token (shown once); old token revoked
GET	/devices/:id/telemetry?from&to&slave&status&page&size	Telemetry log (+ CSV with Accept: text/csv)
GET	/telemetry?device&meter&status&from&to&page&size	Telemetry logs for all devices (Device Management)
GET	/catalogue/device-models · /catalogue/meter-models	Hardware catalogue for the Make & model lists
GET · POST	/register-maps · /register-maps/:id	Register maps; POST imports a maker's list (CSV)
GET · POST	/devices/:id/commands	Command history / send a command
GET	/devices/:id/commands/:cmdId	Status and reply
GET · POST	/devices/:id/publish-destinations	
PATCH · DELETE	/devices/:id/publish-destinations/:destId	Edit, enable / pause, delete
POST	/devices/:id/publish-destinations/:destId/test	Test publish now
GET	/devices/:id/publish-destinations/:destId/preview	Next payload
GET	/devices/:id/data-logs?from&to&kind&status&q&page&size	Publishes + command replies
GET	/devices/:id/events	Status history
GET	/meters?view=tree&site=&status=&kind=	Meter tree (replaces ED_METERS)
POST	/meters	Create main meter
POST	/meters/:id/sub-meters	Create sub-meter
GET · PATCH · DELETE	/meters/:id	Delete = archive
GET	/meters/:id/live	Latest values from the live cache

Method	Path	Purpose
GET	/meters/:id/readings?from&to&interval=auto 1m 15m 1h 1d¶ms=kw,pf	Chart series
GET	/meters/:id/summary?period=today month	kWh, cost, peak, PF, Σ/Δ , import / export energy
GET	/meters/:id/trends?from&to¶ms=kw,kva,pf,thd_v1,thd_v2,thd_v3,thd_i1,thd_i2,thd_i3	Trends tab charts (15-minute)
GET · PUT	/connections/flow	Current draft
POST	/connections/flow/validate · /deploy	
GET	/connections/flow/versions	
POST	/connections/flow/versions/:v/rollback	

Analytics, tariffs, shifts

Method	Path	Purpose
GET	/dashboard/summary	Tiles on the Dashboard
GET	/analytics/consumption?from&to&groupBy=meter site shift day	
GET	/analytics/sub-meter-report?from&to	Sub-meter consumption report
GET	/analytics/demand?month=	15-min maximum demand vs contract
GET	/analytics/power-quality?from&to	PF, THD, voltage, imbalance
GET · PUT	/energy-targets/:meterId/:month	Target & allocation
GET · POST · PATCH · DELETE	/tariffs[:id]	ToD slots
GET	/tariffs/active?at=	Slot and rate now
GET	/tariff-instances?from&to&slot&status&page&size	Tariff instances list
GET	/tariff-instances/summary?from&to	Totals and By-slot table
GET	/tariff-instances/:id	Detail with per-meter split
GET · POST · PATCH · DELETE	/shifts[:id] · /shift-schedules[:id]	
GET · POST · DELETE	/shift-assignments	Assign staff
GET	/shift-instances?from&to	
GET	/shift-instances/:id/energy	Energy, cost, peak per meter

Alerts, notifications, exports, audit

Method	Path	Purpose
GET	/alerts?status&severity&category&search&page&size	Alert log
POST	/alerts/:id/acknowledge · /resolve	{note}
POST	/alerts/acknowledge-all	
DELETE	/alerts/:id	Tenant Admin only
GET · POST	/alert-rules?meter=	
GET · PUT · DELETE	/alert-rules/:id	
POST	/alert-rules/:id/test	24-hour dry run
GET	/notifications?unread=true	Bell inbox
POST	/notifications/:id/read · /read-all	
GET · PUT	/me/notification-preferences	
POST	/exports	{kind, format, from, to, meters}

Method	Path	Purpose
GET	/exports/:id · /exports/:id/download	
GET	/audit?entity&actor&from&to	Tenant Admin, Auditor

Device ingestion (device credentials, not user tokens)

Method	Path	Purpose
POST	/ingest/v1/readings	Batch of readings (HTTPS push gateways)
POST	/ingest/v1/heartbeat	Status, signal, firmware
POST	/internal/mqtt/auth · /internal/mqtt/ac1	Called by EMQX to authorise devices (internal network only)

6.2 Example: /auth/me

```
{
  "user": { "id": "4b1...", "name": "Vijay Kumar", "username": "vijay", "role": "Tenant Admin", "empId": "EMP-001" },
  "tenant": { "id": "FEBINO", "name": "Febino Solutions Pvt Ltd", "status": "active", "timezone": "Asia/Kolkata" },
  "permissions": { "meters": ["view", "create", "edit", "delete", "export"], "users": ["view", "create", "edit"] },
  "fieldPerms": { "meters.cost": { "view": true, "edit": true } },
  "limits": { "meters": { "limit": 50, "used": 12 }, "subMeters": { "limit": 250, "used": 38 } },
  "memberships": [ { "orgId": "GREENTECH", "name": "GreenTech Campus" } ]
}
```

7. Realtime data flow



Figure 2 — Life of one reading.

7.1 Device ingestion over MQTT

Each device connects to the broker over TLS (port 8883) with its client ID as username and its **device token** as password. The broker asks the API whether the device may connect and which topics it may use, so a device can only publish for itself.

Topic	Direction	Content
ed/{orgId}/{deviceCode}/readings	device → cloud	Batch of readings (QoS 1)
ed/{orgId}/{deviceCode}/status	device → cloud	online / heartbeat; broker sends 'offline' as Last Will (retained)
ed/{orgId}/{deviceCode}/cmd	cloud → device	config: polling interval, slave list, firmware URL (later)

```
// ed/FEBINO/GW-01/readings
{
  "ts": "2026-10-08T05:15:00Z",           // gateway clock (NTP); server adds received_at
  "seq": 18452,                          // per-device counter: gaps reveal lost batches
  "readings": [
    { "slave": 1, "kw": 67.1, "kvar": 38.9, "kva": 77.6, "pf": 0.86,
      "v": [232.0, 230.1, 231.9], "v11": [401.2, 399.8, 401.6], "i": [118.2, 112.4, 109.3], "in": 7.8,
      "hz": 50.02, "thd_v": 2.4, "thd_i": 6.1, "kwh_imp": 1284520.4, "kwh_exp": 0 },
    { "slave": 2, "ts": "2026-10-08T05:14:58Z", "kw": 9.7, "pf": 0.95, "v": [231.0], "i": [44.2],
      "hz": 49.96, "kwh_imp": 210388.1 }    // 1-phase meter: one voltage and one current
  ]
}
```

Devices that use **HTTPS push** post the same JSON to /ingest/v1/readings with header Authorization: Bearer <device token>. Devices whose uplink is **Modbus TCP (polled)** are read by the poller: every polling interval it opens

the gateway's IP:port, reads each slave with the meter model's register map and produces the same normalised reading. An on-site Node-RED (with a Modbus node and an MQTT out node) is a good low-cost edge option that publishes exactly this format.

7.2 Processing steps

Step	What happens	On failure
Authenticate	Device identity from MQTT client ID / HTTPS signature; tenant comes from the device, not the payload	Reject, count in metrics
Validate	Schema check (zod); numbers finite; pf in $-1...1$; hz 45–55; voltages $0-1.5 \times$ nominal	Drop field, set quality bit 2
Map	(device, slave) → meter from the Redis device map (rebuilt on flow deploy / meter change)	Unknown slave → device_events warning
Normalise	Apply scaling, put phases in order, 1 ϕ meters keep only v1/i1, derive kva = kw / pf if missing	
Register check	kwh_imp lower than last value → register reset/rollover; energy delta treated as 0 and quality bit 4 set	Raise info alert once
Time	Use device ts; if more than 5 min in the future use received_at; older than 7 days → rejected; late data accepted (aggregates refresh 2 h back)	Set quality bit 8
Publish	XADD to Redis Stream 'readings' (capped ~1 M entries); update device last_seen	Retry; broker keeps QoS 1 message
Log	Every message, good or bad (timeouts and CRC errors reported by the gateway), is written to device_messages for the device's Telemetry log	

7.3 Stream consumers

- **db-writer** group → TelemetryService.writeBatch: COPY into readings; ON CONFLICT DO NOTHING handles duplicates.
- **live** group → LiveService: updates live:{org}:{meter}, recomputes the parent's Σ / Δ , emits WebSocket events.
- **alerts** group → AlertEngine: evaluates live rules for that meter (and its parent's rules that cover sub-meters).
- Each group acknowledges entries only after success (XACK); a crashed worker's pending entries are re-claimed by another (XAUTOCLAIM).

7.4 WebSocket protocol (browser)

Socket.IO namespace /live. The browser connects with its access token; the server puts the socket in the room t:{orgId} and joins further rooms only for things the user may see.

Event	Direction	Payload / meaning
subscribe	client → server	{ topics: ['dashboard', 'meter:MTR-1001', 'device:GW-01', 'alerts'] }
unsubscribe	client → server	same shape; sent when leaving a page
reading	server → client	{ meter, ts, kw, kva, pf, v, i, hz, thd_v, thd_i, load, kwhToday, cost?, sum?, delta? } — max 1/s per meter
meter.status	server → client	{ meter, status: online idle offline maintenance generating }
device.status	server → client	{ device, status, signal, lastSeen }
dashboard	server → client	{ liveKw, todayKwh, todayCost, co2, maxDemand, peak } every 5 s
alert	server → client	{ type: raised acknowledged resolved, alert } — updates the Alert log and the bell badge
notification	server → client	in-app message for this user
session.revoked	server → client	tenant suspended / user removed → front end signs out

```
// Front end (assets/js/live.js) – replaces the random simulator
const socket = io('/live', { auth: { token: Api.accessToken() } });
socket.on('connect', () => socket.emit('subscribe', { topics: ['meter:' + meterId, 'alerts'] }));
socket.on('reading', r => { if (r.meter === meterId) updateGauges(r); });
socket.on('connect_error', async () => { await Api.refresh(); socket.auth.token = Api.accessToken(); });
```

7.5 Online, idle and offline

Status	Rule
Device online / weak	last message within 3 × polling interval; weak if signal under 40 %
Device offline	nothing for 3 × polling interval (or MQTT Last Will) → device_events + system alert 'Gateway offline'
Meter online	latest reading within 3 × its device's polling interval and kW above 2 % of rated
Meter idle	reporting, but kW at or below 2 % of rated (or under 0.5 kW without a rating)
Meter generating	kind = generator and exporting / producing
Meter offline	device online but this slave has not answered for 3 intervals, or its device is offline
Maintenance	set by a user; no offline alerts while in maintenance

8. Alert engine

The engine runs the rules people build in the meter page's New alert rule form. Rules are stored exactly in that JSON shape, then compiled on save.

8.1 Rule JSON (as saved by the UI)

```
{
  "meter": "MTR-1001", "name": "Low power factor on incomer", "severity": "Warning", "enabled": true,
  "scope": "meter", "subs": [], "evaluate": "live",
  "cond": {
    "mode": "guided", "match": "all", "hold": 5,
    "rows": [ { "param": "pf", "op": "lt", "v1": 0.9 }, { "param": "kw", "op": "gt", "v1": 20 } ],
    "expr": "" },
  "active": {
    "mode": "shifts", "shifts": ["A", "B"], "slots": [], "from": "22:00", "to": "06:00",
    "days": ["Mon", "Tue", "Wed", "Thu", "Fri", "Sat"] },
  "notify": {
    "channels": ["app", "email"], "roles": ["Energy Manager", "Supervisor"], "users": [],
    "emails": [], "repeat": "once", "resolve": "auto5", "escalate": 30 }
}
```

8.2 Parameters and operators

Parameter key	Meaning	1φ meters
kw, kva	Active / apparent power	yes
load	kW as % of rated kW	yes
pf	Power factor	yes
v_min, v_max	Lowest / highest phase voltage	single voltage
v_imb, i_imb	Voltage / current imbalance %	not available
i_max	Highest phase current	single current
hz	Frequency	yes
thd_v, thd_i	Harmonic distortion %	yes
kwh, cost	Energy / cost in the evaluation window	yes
nodata	Minutes without readings (only 'is above')	yes

Operators: gt is above, lt is below, between, outside, rise rises by more than (vs. the value hold minutes ago), drop drops by more than. **Expression mode** allows formulas over the variables kw, kva, load, pf, v1, v2, v3, i1, i2, i3, v_imb, i_imb, hz, thd_v, thd_i, kwh, cost (1φ meters: only v1 and i1, no imbalance). Expressions are parsed with a whitelisted expression library (for example expr-eval) — **never** with eval or new Function. Unknown

variables or functions are rejected when the rule is saved.

8.3 State machine

From	Event	To	Action
ok	condition true and inside an active window	pending	remember since
pending	still true for hold minutes	firing	create alert, notify recipients
pending	becomes false	ok	nothing
firing	repeat interval passes, still true	firing	re-notify (repeat: once 15m 1h)
firing	not acknowledged after 'escalate' minutes	firing	notify Tenant Admin(s), escalated_at
firing	false for 5 min (resolve: auto5)	ok	resolve alert automatically
firing	user resolves (resolve: manual)	ok	resolve with note
any	outside active window / rule disabled / meter in maintenance	ok	pending cleared; open alert stays until resolved

8.4 Evaluation details

- **Scope:** meter → the meter itself; subs → each selected sub-meter has its own state; meter_and_sub → both.
- **Windows** (kwh, cost, or evaluate = 15m/1h/shift/day) are checked when the window closes, using readings_15m / readings_1h / the shift instance.
- **Active windows** use the site's time zone: shifts resolve overnight runs (22:00 → 06:00 belongs to the start day unless count_on = end_day); ToD slots use the tariff table; custom from/to may cross midnight; days filter by weekday.
- **Missing data** never counts as 'true' except for the nodaata parameter, which the offline checker feeds every 30 s.
- **State** lives in Redis for speed (ars: {rule}: {meter}) and is written to alert_rule_state on every transition, so a restart resumes correctly.
- **Throughput:** rules are indexed by meter in memory; one reading touches only the rules of that meter and its parent.

9. Calculations

Value	Formula / rule
Current (estimate, 3φ)	$I = P \times 1000 / (\sqrt{3} \times V_{LL} \times PF)$ — per-phase current from total kW
Current (estimate, 1φ)	$I = P \times 1000 / (V_{LN} \times PF)$
Apparent power	kVA = kW / PF (or $\sqrt{(kW^2 + kvar^2)}$); measured values from the meter are preferred over formulas
Load %	kW / rated_kw × 100
Imbalance %	max phase – average / average × 100 (voltage and current, 3φ only)
Energy in a period	last(kwh_imp) – first(kwh_imp) from the meter's own register — never by adding up kW samples; resets handled as 0
Maximum demand	highest 15-minute average kVA (and kW) in the billing month, compared with the contract demand
Cost	Σ (energy in each ToD slot × slot rate) + demand charge × max demand; tenant default rate when no slot matches
CO2	kWh × tenant co2_factor (default 0.82 kg/kWh)
Σ sub-meters / Δ unmetered	Σ = sum of sub-meters' kW (or kWh); Δ = main – Σ ; Δ % = Δ / main × 100 (shown as 'unmetered')
Virtual meter	Σ sign × member value, evaluated in the live service and in queries
Shift energy	energy of each meter between the shift instance's starts_at and ends_at

10. Security

Area	Design
Passwords	argon2id (memory 64 MB, 3 iterations); minimum 8 chars with letters and numbers; breached-password check optional
Tokens	Access JWT 15 min (sub, tid, rid, realm, sid); refresh token random 256-bit, stored hashed, rotated on every use, reuse → whole session revoked
Cookies	Refresh cookie httpOnly, Secure, SameSite=Strict, path /api/v1/auth; CSRF token for cookie-based refresh
Tenant isolation	Every query runs with SET app.tenant_id; PostgreSQL row-level security policies on all tenant tables; tenant comes from the token only
Permissions	Module/action guard on every route; field permissions filter responses and reject forbidden writes; Tenant Admin always full
Platform realm	Separate login, 8-hour session, MFA, IP allow-list optional; every action audited
One-time codes	6 digits, hashed, 10 minutes, 5 attempts then locked; 60 s resend cooldown; rate-limited per IP and per target
Rate limits	Login 10/min per IP and per account; API 600/min per user; ingest per device by polling interval
Devices	One device token per device (shown once, stored hashed, regenerable), TLS only, topic ACLs per client ID
Data	TLS everywhere; encrypted disks and backups; secrets in environment / vault, never in the repo; PII masked in logs
Suspension	Suspending a tenant or user revokes sessions and pushes session.revoked over WebSocket
Backups	Daily full + continuous WAL archiving (point-in-time restore 7 days); restore tested monthly

11. Connecting the existing front end

Add one file, `assets/js/api.js` (fetch wrapper: base URL, bearer token, automatic refresh on 401, error mapping) and `assets/js/live.js` (Socket.IO client). Keep `window.EDStore` as a façade so pages change as little as possible, but make its functions asynchronous and call the API.

Today (browser demo)	Becomes
<code>EDStore.findLogin + checkPassword + signIn</code>	<code>POST /auth/login</code>
<code>EDStore.session()</code>	<code>GET /auth/me</code> (cached for the page)
<code>EDStore.signOut / switchTenant</code>	<code>POST /auth/logout / POST /auth/switch-tenant</code>
<code>startPending / checkCode / resendPending</code>	<code>POST /auth/otp/request · /verify, /auth/verify-email</code>
<code>createTenant (register.html)</code>	<code>POST /auth/register</code>
<code>tenants / setTenantStatus / setLimits</code>	<code>/admin/tenants ... approve · suspend · limits</code>
<code>users / createUser / updateUser / deleteUser</code>	<code>GET · POST /users, PATCH · DELETE /users/:id</code>
<code>findInvite / acceptInvite</code>	<code>GET · POST /invites/:token[/accept]</code>
<code>fieldPerms / setFieldPerms / canField</code>	<code>/roles/:id/field-permissions; canField reads /auth/me</code>
<code>limits / limitReached</code>	<code>GET /tenant/limits</code> (server also returns 409 LIMIT_REACHED)
<code>meterRules / saveMeterRule / deleteMeterRule</code>	<code>/alert-rules</code>
<code>flow / saveFlow (connections.html)</code>	<code>GET · PUT /connections/flow, POST /deploy</code>
<code>window.ED_METERS (meter-data.js)</code>	<code>GET /meters?view=tree</code>
<code>window.ED_DEVICES (device-data.js)</code>	<code>GET /devices</code>
Random live values (<code>meter.js</code> , <code>dashboard.js</code>)	<code>GET /meters/:id/live + WebSocket 'reading'</code>
Generated chart history	<code>GET /meters/:id/readings</code>
<code>export.js</code> (browser CSV/PDF)	Same for small ranges; <code>POST /exports</code> for large
<code>pager.js</code> (client paging)	Kept for short lists; Users, Alerts and readings switch to server paging (page, size)

Pages need to be served from the same origin as the API (Nginx serves the static files and proxies /api and /socket.io), so cookies and CORS stay simple.

12. Project structure

```

backend/
├── src/
│   ├── main.ts                API process (REST + WebSocket)
│   ├── workers/
│   │   ├── ingest.worker.ts   MQTT consumer, Modbus poller
│   │   ├── stream.worker.ts   db-writer, live, alerts consumer groups
│   │   └── jobs.worker.ts      notifications, exports, schedulers
│   ├── common/
│   │   ├── guards (jwt, tenant, permission), field-permission interceptor,
│   │   ├── pagination, error filter, validation, idempotency
│   │   └── typed env config
│   ├── config/
│   ├── database/              prisma.service.ts, rls.middleware.ts, sql/ (Timescale migrations)
│   └── modules/
│       ├── auth/ platform/ tenants/ users/ roles/ sites/
│       ├── devices/ meters/ connections/
│       ├── ingestion/ telemetry/ live/ realtime/
│       ├── analytics/ tariffs/ shifts/
│       ├── alerts/ (rules, engine/, expression/, alert-log)
│       ├── notifications/ (email, sms, whatsapp, webhook, in-app)
│       └── exports/ audit/ scheduler/
├── prisma/schema.prisma       relational tables
├── prisma/seed.ts             demo tenants, users, meters, devices (same data as the UI demo)
├── tools/simulator/           publishes realistic MQTT readings for all demo meters
├── test/                      unit + e2e (testcontainers: Postgres, Redis, EMQX)
├── docker-compose.yml
└── .env.example
  
```

12.1 Environment variables

Variable	Example / note
DATABASE_URL	postgres://ed:***@db:5432/energy
REDIS_URL	redis://redis:6379
MQTT_URL, MQTT_INTERNAL_USER, MQTT_INTERNAL_PASS	mqtt://broker:8883 (service account for consumers)
JWT_ACCESS_SECRET, JWT_REFRESH_SECRET	long random strings (or an RS256 key pair)
APP_URL	https://app.example.com — used in invite / reset links
SMTP_HOST, SMTP_USER, SMTP_PASS, MAIL_FROM	or SES credentials
SMS_PROVIDER, SMS_API_KEY, WHATSAPP_API_KEY	
S3_ENDPOINT, S3_BUCKET, S3_KEY, S3_SECRET	export files
RAW_RETENTION_DAYS	90
DEFAULT_TIMEZONE	Asia/Kolkata

12.2 Local environment (docker-compose)

```

services:
  db: { image: timescale/timescaledb:latest-pg16, environment: [POSTGRES_PASSWORD=dev], ports: ["5432:5432"] }
  redis: { image: redis:7, ports: ["6379:6379"] }
  broker: { image: emqx/emqx:5, ports: ["1883:1883", "8883:8883", "18083:18083"] }
  minio: { image: minio/minio, command: server /data, ports: ["9000:9000"] }
  api: { build: ., command: node dist/main.js, depends_on: [db, redis, broker] }
  stream: { build: ., command: node dist/workers/stream.worker.js, depends_on: [db, redis] }
  ingest: { build: ., command: node dist/workers/ingest.worker.js, depends_on: [broker, redis] }
  jobs: { build: ., command: node dist/workers/jobs.worker.js, depends_on: [db, redis] }
  web: { image: nginx, volumes: ["/usr/share/nginx/html:ro", "/.nginx.conf:/etc/nginx/conf.d/default.conf"],
        ports: ["8080:80"] } # serves the existing pages, proxies /api and /socket.io
  
```

13. Build plan

Indicative order for a team of two to three developers. Each phase ends with the front end using the new backend for that area, so progress is visible early.

Phase	Deliverables	Front end switches to API	Indicative
1 · Foundation	Repo, Docker Compose, Postgres + Timescale, Redis, NestJS skeleton, config, error/paging conventions, audit; Auth, tenants, users, invites, roles, field permissions, limits; Control Center	login, register, approvals, users, roles, settings (profile, sessions)	2–3 weeks
2 · Assets	Sites, devices (+ credentials), meters/sub-meters (phase, CT, rated kW), register maps, connection flow deploy, seed data	meters, devices, device, connections	1–2 weeks
3 · Realtime	EMQX, ingest worker, Modbus poller, readings hypertable + aggregates, live service, WebSocket gateway, simulator for all demo meters, offline detection	dashboard, meter page live values and charts	2 weeks
4 · Analytics & operations	Tariffs and cost, max demand, analytics endpoints, shifts, schedules, instances with energy	analytics, shifts, schedule, instances, tariffs	1–2 weeks
5 · Alerts	Rule compile + validation, engine, state machine, alert log, notifications (app, email, SMS/WhatsApp, webhook), escalation, dry-run	alerts, meter alert rules, bell	2 weeks
6 · Hardening	Exports, rate limits, RLS tests, load test (e.g. 5,000 meters at 15 s), monitoring dashboards, backups, production deploy	exports	1–2 weeks

13.1 Definition of done for 'realtime'

- A reading published by the simulator appears on the meter page in under 2 seconds (p95), measured from ts to browser render.
- Stopping a simulated gateway marks it offline within 3 polling intervals and raises a 'Gateway offline' alert.
- A rule 'PF below 0.9 for 5 minutes' fires within 5 minutes + 15 s of the condition starting, notifies by app and email, and auto-resolves 5 minutes after it clears.
- Two users of the same tenant see the same values; a user of another tenant cannot read them (verified by automated RLS tests).
- Restarting any worker loses no readings (stream re-delivery) and keeps alert states.

13.2 Testing

Level	What
Unit	Formulas (phase currents, imbalance, energy with resets, ToD split across midnight), rule evaluator and state machine, expression validation, limits
Integration	Repositories against a real Postgres/Timescale (testcontainers); RLS: tenant A can never read tenant B
End-to-end	Simulator → MQTT → ingest → DB/live/alerts → WebSocket client receives event; auth flows incl. OTP lock-out and suspension
Load	k6 / MQTT bench: target 5,000 meters at 15 s ($\approx$ 333 readings/s) with ingest lag under 1 s